An Integrated Green DevOps Framework for Sustainable Software Delivery and Operations

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
Mendis J.D.L.	IT22110084	Release Stage
Rathnayake D.M.B.H.	IT22149930	Deploy Stage
Thassara M.P.M.	IT22281432	Operate Stage
Kodithuwakku I.P.	IT22189776	Monitor Stage

Report submitted in partial fulfilment of the requirements for the
B.Sc. (Hons) Degree in Information Technology
Specialization in
Software Engineering
Department of Software Engineering
Sri Lanka Institute of Information Technology
Sri Lanka
October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
Mendis J.D.L.	IT22110084		
Rathnayake D.M.B.H.	IT22149930		
Thassara M.P.M.	IT22281432		
Kodithuwakku I.P.	IT22189776		

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Jagath Wickramarathne		
Co-supervisor: Ms. Madusha Weerasooriya		

ABSTRACT

Software delivery pipelines and cloud-native platforms are commonly optimized for speed, reliability, and resource availability, while energy use, grid carbon intensity, and sustainability regressions are rarely considered as coordinated decision inputs across the delivery lifecycle. This report presents an integrated Green DevOps framework that connects four components across the Release, Deploy, Operate, and Monitor stages. The Release component applies change-impact analysis, selective build and test execution, and carbon-aware scheduling; Deploy uses carbon context and deployment-state information to support bounded release timing and strategy decisions; Operate combines LSTM-based workload forecasting, software-based energy and carbon estimation, SLA-aware scaling, and workload prioritisation; and Monitor establishes stage-specific baselines, anomaly detection, and a consolidated Sustainability Health Score. The prototype integrates these components through shared carbon context, structured hand-offs, runtime telemetry, and a feedback path from monitoring to earlier stages. Evaluation produced feasibility-level evidence: Release reduced mean pipeline time by 45.3% across 11 paired runs and skipped 91.3% of test executions; Deploy recorded 17 deploy and 13 wait decisions, with 11 waits followed by a greener window; Operate achieved an LSTM R^2 of approximately 0.95 with a MAPE of 3.28%; and Monitor analysed 63 stage-level records using statistical and machine-learning detectors. The results indicate that carbon-aware decisions can be integrated across the software delivery lifecycle while retaining bounded delays, SLA protection, and fallback behaviour. The evidence remains limited to a single-VM, single-K3s-cluster, single-grid-zone prototype and should therefore be interpreted as a demonstration of feasibility rather than a production-scale estimate of aggregate carbon reduction.

Keywords: Green DevOps, Carbon-Aware Computing, CI/CD, Kubernetes, Sustainable Software

ACKNOWLEDGEMENTS

We express our sincere gratitude to our supervisor, Mr. Jagath Wickramarathne, and our co-supervisor, Ms. Madusha Weerasooriya, of the Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT), for their guidance throughout this project. Their feedback on the problem definition, the design of the integrated framework and the evaluation helped us keep the work focused and rigorous.

We are also grateful to our external supervisor,Mr. Dilanka Nayanajith, for his Industrial advice on DevOps practice and the needs of practitioners. His input helped us ground the framework in the concerns of real delivery teams.

We thank the Faculty of Computing for the facilities and academic environment that made this work possible.

This research depended on publicly available data and open tools. We acknowledge the providers of the Bitbrains fastStorage workload traces, Electricity Maps for grid carbon-intensity data, and the national system operator for published generation statistics for Sri Lanka. We are also grateful to the open-source communities behind Jenkins, Maven, K3s, Prometheus, Grafana, LightGBM, Prophet and Ollama, on which the prototype is built.

Finally, we thank our families and friends for their support during the project.

TABLE OF CONTENTS

1. INTRODUCTION AND PROJECT CONTEXT 10
1.1. Background and context 10
1.2. Stakeholders and user needs 10
1.3. Alignment with the Sustainable Development Goals 11
1.4. Purpose and scope of the common report 12
2. OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 13
2.1. Overall problem definition 13
2.2. Root causes and overall research gap 14
2.2.1. Root causes 14
2.2.2. Limitations of current approaches 14
2.2.3. Overall research gap 15
2.3. Subproblem and component mapping 15
2.3.1. Subproblem 1: redundant and carbon-blind build and test execution (Release) 16
2.3.2. Subproblem 2: carbon-blind deployment timing and strategy (Deploy) 17
2.3.3. Subproblem 3: reactive, carbon-unaware and workload-blind scaling (Operate) 17
2.3.4. Subproblem 4: measurement without sustainability intelligence (Monitor) 18
2.3.5. Interdependence and necessity of all four subproblems 18
2.4. Overall aim and group objectives 19
2.5. Scope, assumptions and constraints 20
2.5.1. Scope and exclusions 20
2.5.2. Assumptions 21
2.5.3. Constraints 21
2.5.4. Dependencies 22
3. OVERALL SOLUTION STRUCTURE 23
3.1. Integrated solution overview 23
3.2. Component 1 summary 26
3.3. Component 2 summary 26
3.4. Component 3 summary 27
3.5. Component 4 summary 28
3.6. End to end user and data flow 28

4. COMPONENT INTEGRATION AND COHESIVE OPERATION 32
4.1. Integration strategy 32
4.2. Interfaces data contracts and shared models 34
4.3. Shared nonfunctional requirements 34
4.4. Integrated workflow and exception handling 36
4.4.1. Normal end-to-end workflow 36
4.4.2. Failure paths and fallbacks 38
4.4.3. Fallback hierarchy 40
4.4.4. Bounds and guarantees 41
4.4.5. Recovery and logging 42
4.4.6. Cross-component behaviour and the feedback path 42
4.4.7. Residual gaps and risks 42
4.5. Team contribution and dependency matrix 43
4.5.1. Individual contributions 43
4.5.2. Shared and integration work 44
4.5.3. Dependency matrix 45
4.5.4. How contributions were verified 46
5. INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 47
5.1. Common technology stack and infrastructure 47
Shared platform and infrastructure 47
Technology added by each component 48
Repositories and data stores 48
Environments 49
Reasons for the common choices 49
5.2. Development and integration process 49
5.3. Deployment and operational configuration 50
5.4. Evidence of team execution 52
6. INTEGRATED TESTING EVALUATION AND VALIDATION 54
6.1. Evaluation framework and success criteria 54
6.2. Integration and end to end testing 54
6.3. Overall results 54
6.4. Stakeholder and user validation 54
6.5. Achievement of common objectives 54
6.6. Integrated limitations and threats 56
6.6.1. System-level limitations 56

6.6.2. Dependencies between components 56
6.6.3. Threats to validity 57
6.6.4. Unresolved integration risks 58
7. COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION 59
7.1. Value proposition and beneficiaries 59
7.2. Market alternatives and differentiation 60
7.3. Business model and feasibility 61
7.4. Adoption, intellectual property and compliance 62
7.5. Sustainable Development Goal contribution 63
8. CONCLUSION AND FUTURE DIRECTIONS 64
8.1. Overall conclusion 64
8.2. Principal integrated contribution 64
8.3. Future directions 65
REFERENCES 67
APPENDICES 69
Appendix A Integrated interface specification 69
Appendix B Team contribution declaration 69

LIST OF FIGURES

Figure 2.1 Decomposition of the overall problem into four lifecycle subproblems .. 16
Figure 3.1 High level architecture..... 25

LIST OF TABLES

Table 1.1 Stakeholders and their needs 11
Table 1.2 Alignment of the project with the SDGs 11
Table 2.1 Root causes of the overall problem 14
Table 2.2 Distinguishing features of the four subproblems 16
Table 2.3 Interdependence of the subproblems and consequence of omitting each 18
Table 2.4 Group objectives, components and evaluation evidence 19
Table 2.5 Scope boundaries of the integrated solution 20
Table 3.1 The integrated framework at a glance 23
Table 3.2 Data items moving through the framework 30
Table 4.1 Integration sequence, interfaces and ownership 32
Table 4.2 Integration risks and how they were managed 33
Table 4.3 Shared nonfunctional requirements 34
Table 4.4 Controls that alter the normal workflow 37
Table 4.5 Failure paths, fallbacks and effect on delivery 38
Table 4.6 Limits that keep the framework from blocking delivery 41
Table 4.7 Individual contributions by component 43
Table 4.8 Shared and integration tasks 45
Table 4.9 Dependency matrix (row depends on column) 45
Table 5.1 Shared platform, services and tooling 47
Table 5.2 Component-specific technology 48
Table 5.3 Main configuration items 51
Table 5.4 State and recovery 52
Table 5.5 Sources of execution evidence 52
Table 5.6 Integration milestones. 53
Table 6.1 Achievement of the group objectives 54
Table 6.2 Inter-component dependencies and their consequences 56
Table 6.3 Unresolved integration risks 58
Table 7.1Beneficiaries and the value created 59
Table 7.2 Alternatives and substitutes 60
Table 7.3 Alternatives and substitutes 61
Table 7.4 Adoption barriers and mitigation 62
Table 7.5 Contribution to selected SDG targets 63
Table 8.1 Prioritised future work 65

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
CSV	Comma-Separated Values
DORA	DevOps Research and Assessment
HPA	Horizontal Pod Autoscaler
LSTM	Long Short-Term Memory
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
R^2	Coefficient of Determination
SCI	Software Carbon Intensity
SDG	Sustainable Development Goal
SLA	Service-Level Agreement
VM	Virtual Machine

1. INTRODUCTION AND PROJECT CONTEXT

1.1. Background and context

The digitalization of the global economy has created a large and growing environmental burden. The information technology sector accounts for roughly 1.8–3.9% of global greenhouse gas emissions [1], [2], and data centers alone are estimated to consume about 70 billion kWh of electricity a year, a figure expected to rise as AI-driven infrastructure expands [3]. Much of this footprint is hidden in the software lifecycle. Nearly half of the sector's emissions come from hardware manufacturing [3], while modern Continuous Integration and Continuous Deployment (CI/CD) pipelines add to energy use through frequent automated build, test and deployment activity [4], [5]. Sustainability is therefore increasingly treated as a software quality attribute alongside reliability, maintainability and performance [6], which has driven work on sustainable software engineering [7], [8]. Green DevOps extends this idea by building environmental responsibility into DevOps practice, adding an environmental dimension to delivery metrics such as the DORA indicators [3]. Cloud-native infrastructure adds to the pressure, because Kubernetes now automates deployment and scaling at a scale that is itself a significant driver of data-center electricity demand [9], [10].

Wasteful behavior appears at every stage of the software delivery lifecycle, and each stage has its own character. At the Release stage, CI pipelines usually rebuild and re-test a whole multi-module repository on every commit, whatever the scope of the change, and they ignore how grid carbon intensity varies through the day [11], [12]. At the Deploy stage, software is normally promoted to production as soon as the build finishes, even though carbon-aware research shows that moving flexible work to cleaner periods can reduce emissions [13], [14]. At the Operate stage, Kubernetes autoscaling is mostly reactive and relies on CPU and memory thresholds. It responds only after demand has risen, takes no account of the environmental cost of scaling, and treats all workloads alike [9], [15]. At the Monitor stage, observability platforms report performance and resource use but give little help in establishing sustainability baselines or detecting regressions in environmental behavior [16], [17].

Measurement standards do exist. The Software Carbon Intensity (SCI) specification, for example, gives a method for estimating software emissions from energy use and grid carbon intensity [18]. However, research on measurement, test selection, carbon-aware scheduling, predictive scaling and anomaly detection has mostly treated these concerns separately, and Chapter 2 defines the resulting gap formally. This project responds with one integrated Green DevOps framework. Four components, one for each lifecycle stage (Release, Deploy, Operate and Monitor), share carbon-aware information through a closed feedback loop. The prototype was developed and evaluated using carbon-intensity data for the Sri Lankan grid, which grounds the work in a local setting.

1.2. Stakeholders and user needs

Table 1.1 groups the stakeholders of the framework by role and lists the needs that follow from the problem analysis in Section 1.1.

Table 1.1 Stakeholders and their needs

Type	Stakeholder	Needs	Components
Primary	DevOps and platform engineers	Pipelines that finish reliably with less wasted compute; carbon-aware scheduling that does not block delivery; low-effort setup inside existing tools	Release, Deploy
Primary	Software developers	Fast feedback on each commit; selective builds that still catch failures	Release
Primary	Site reliability and operations engineers	Scaling that meets service-level agreements (SLAs) while avoiding over-provisioning; deferral of delay-tolerant jobs in high-carbon periods	Operate
Primary	Engineering managers and sustainability officers	A trustworthy sustainability indicator and early warning of regressions	Monitor
Secondary	Cloud and infrastructure providers	Efficient resource use and lower energy demand	Deploy, Operate
Secondary	Organisations and wider society	Lower emissions and cost; support for sustainability commitments	All
Secondary	Academic community and supervisors	A reproducible method with transparent evidence	All

Delivery speed and service levels are the main concerns of the primary stakeholders, so the framework treats them as constraints rather than trade-offs. Carbon-aware delays are bounded, and scaling decisions are SLA-aware.

1.3. Alignment with the Sustainable Development Goals

The project supports the United Nations Sustainable Development Goals (SDGs) [19] summarised in Table 1.2. Its primary alignment is with SDG 13 (Climate Action), and it also contributes to SDG 7, SDG 9 and SDG 12.

Table 1.2 Alignment of the project with the SDGs

SDG	Intended contribution	Components
SDG 13: Climate Action	Reduces unnecessary computation and shifts flexible work to lower-carbon periods, lowering the emissions of software delivery	All
SDG 12: Responsible Consumption and	Avoids redundant builds, tests and over-provisioned resources, so computing resources	Release, Operate

SDG	Intended contribution	Components
Production	are used more efficiently	
SDG 9: Industry, Innovation and Infrastructure	Provides tooling that makes software delivery infrastructure more efficient and sustainability-aware	Release, Deploy, Monitor
SDG 7: Affordable and Clean Energy	Aligns energy-intensive pipeline and deployment work with periods of cleaner grid supply	Release, Deploy

These contributions are intended effects at pipeline and cluster level. The reported results come from a prototype evaluated on a single virtual machine, one K3s cluster and one grid zone (Sri Lanka), so this report does not claim measured effects on national or global emissions. The framework can also have side effects. Delayed builds could slow developer feedback, which is why delays are bounded, and the framework's own models consume energy, which is why overhead is measured (for example, the Release analysis adds about one second per pipeline execution).

1.4. Purpose and scope of the common report

This common report presents the integrated problem and the integrated solution as a whole: how the four components fit together, how they were implemented and evaluated as one system, and what value the complete solution offers. The four individual theses provide the detailed component-level research, development and evidence:

- Release: git-diff selective build and test execution with a LightGBM carbon-intensity classifier.
- Deploy: a ReAct-based AI agent for carbon-aware deployment strategy selection and release-window scheduling.
- Operate: LSTM workload forecasting, carbon-emission estimation and SLA-aware workload prioritisation for Kubernetes scaling.
- Monitor: hybrid statistical and Isolation Forest anomaly detection with a consolidated Sustainability Health Score.

To keep the report focused on the complete solution, component methods and experiments are summarised only as far as needed to explain integration, and readers are referred to the individual theses for detail. The scope of the evaluated prototype is limited to Maven-based multi-module projects in Jenkins pipelines, a single K3s cluster, workload traces from the Bitbrains fastStorage dataset, and carbon-intensity data for the Sri Lankan grid.

The remainder of the report is organised as follows. Chapter 2 defines the overall problem and the four subproblems. Chapter 3 describes the overall solution structure and summarises each component. Chapter 4 explains how the components are integrated, and Chapter 5 describes the implementation and project execution. Chapter 6 presents integrated testing, evaluation and validation. Chapter 7 discusses commercialisation, sustainability and responsible innovation, and Chapter 8 concludes the report and outlines future directions.

2. OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1. Overall problem definition

Chapter 1 described how the environmental cost of software is spread across the delivery lifecycle. This section condenses that background into one problem statement. The statement is deliberately kept apart from any proposed technology, so that the solution in Chapter 3 can be judged against it.

> **Overall problem.** Software delivery pipelines and the cloud-native platforms they deploy to are typically governed by delivery speed, reliability and resource availability alone. At no stage of the lifecycle (release, deployment, operation or monitoring) is the energy used, or the carbon intensity of the electricity supplying it, normally treated as a decision input, and increases in environmental cost are not detected when they occur. As a result, avoidable computation is performed, flexible work is carried out when the grid is most carbon-intensive, and regressions in environmental behaviour go unnoticed, while teams have no accepted way to reduce these effects without risking delivery velocity or service-level compliance.

The problem is material at sector level. Information technology accounts for roughly 1.8–3.9% of global greenhouse gas emissions [1], [2], CI/CD pipelines add to energy use through frequent automated build, test and deployment activity [4], [5], and Kubernetes-based infrastructure automates scaling at a scale that is itself a driver of data-centre electricity demand [9], [10]. These figures describe the sector, not delivery pipelines alone, and this report makes no claim of a measured effect on sector-level emissions (Section 1.3).

The problem takes a different form at each lifecycle stage. At the Release stage, work is not proportional to the change: whole repositories are rebuilt and retested for every commit [11], [12]. At the Deploy stage, software is promoted as soon as the build ends, whatever the grid is doing [13], [14]. At the Operate stage, scaling reacts after thresholds are crossed, looks only at performance metrics and treats all workloads alike [9], [15]. At the Monitor stage, environmental measurements are collected without baselines against which a regression could be recognised [16], [17].

The remedy is constrained. The primary stakeholders in Table 1.1 care first about delivery speed and service levels. A solution that slows developer feedback, holds back urgent releases or risks service-level agreement (SLA) breaches would not be adopted. The problem is therefore to reduce unnecessary computation and avoidable emissions *within* bounds on delay and service quality, not to trade one for the other without limit. The problem concerns the operational energy and emissions of delivery and runtime activity that a delivery team can influence; its boundaries are set in Section 2.5.

2.2. Root causes and overall research gap

2.2.1. Root causes

Six root causes explain why the problem persists. Four are specific to a lifecycle stage and two cut across all stages. Table 2.1 summarises them with the evidence for each.

Table 2.1 Root causes of the overall problem

ID	Root cause	How it appears in current practice	Stage
RC1	Execution scope is fixed by pipeline configuration, not by what changed	Every commit triggers a rebuild and retest of the whole multi-module repository, so the cost scales with repository size rather than change size [11], [12].	Release
RC2	Timing and strategy decisions ignore grid carbon intensity	Builds start on commit and deployments start on build completion, although grid carbon intensity varies over time and shifting flexible work can lower emissions [13], [14].	Release, Deploy
RC3	Runtime control is reactive, performance-only and workload-blind	Autoscaling responds after thresholds are breached, uses CPU and memory only, ignores the environmental cost of scaling and does not distinguish delay-tolerant jobs [9], [15].	Operate
RC4	Measurement is not coupled to interpretation	Standards define how to estimate emissions [18], and anomaly detection methods are mature [16], [17], but sustainability baselines and regression detection are not part of delivery monitoring (Section 1.1).	Monitor
RC5	Decisions are taken stage by stage, with no shared carbon context or feedback	Test selection, carbon-aware scheduling, predictive scaling and anomaly detection have mostly been studied as separate concerns [12], [14], [15], [16]; one stage's outputs do not inform the next, and monitoring findings do not flow back.	All
RC6	Sustainability is perceived to compete with speed and service levels	Stakeholders prioritise delivery speed and SLAs (Table 1.1); carbon-aware scheduling studies report emission reductions without significant performance impact [14], but this is rarely built into delivery tooling with explicit bounds.	All

2.2.2. Limitations of current approaches

Single-concern techniques are mature but local. Regression test selection and minimisation reduce test effort [11], [12], but they do not consider when execution should happen. Temporal workload shifting lowers emissions for flexible work, including in carbon-aware Kubernetes scheduling [13], [14] and in industrial data-centre practice [20], but it is applied to workloads rather than to the stages of a delivery pipeline. Predictive resource management can improve utilisation [15], yet it is rarely combined with workload-priority-aware decisions in operational Kubernetes settings [9].

Work on sustainable deployment has examined, for example, how microservices can be placed on energy-efficient nodes across the cloud–edge continuum [21]. Established deployment

strategies such as rolling and canary release were designed to reduce deployment risk [22], and choosing a release window or strategy according to grid carbon intensity is not part of that design.

At the framework level, Green and Sustainable DevOps proposals exist, but much of the literature remains strategic or focused on individual practices. Recent reviews identify practical challenges such as limited visibility of energy metrics, overly triggered pipelines, and organisational priorities that favour delivery speed over sustainability [23]. Position work on Sustainable DevOps similarly frames a broader research and practice agenda rather than a complete operational lifecycle mechanism [24]. Measurement standards such as SCI [18] explain how software carbon intensity can be quantified, but they do not prescribe the integrated release, deployment, scaling, and regression-detection decisions implemented in this project.

2.2.3. Overall research gap

The gap is not the absence of any single technique but the absence of an integrated, carbon-aware decision capability across the delivery lifecycle. To the best of the team's knowledge, based on the literature reviewed in the individual theses, three conditions remain unmet:

G1 **Lifecycle coverage.** No framework applies carbon-aware decision-making consistently across the Release, Deploy, Operate and Monitor stages of one delivery pipeline.

G2 **Shared signal and feedback.** No mechanism lets carbon-intensity information and measured outcomes pass between stages, or lets monitoring findings influence earlier release, deployment and scaling decisions.

G3 **Operational evidence.** Lifecycle-wide proposals are largely strategic and unquantified [23], and the use of anomaly detection for sustainability regressions in CI/CD environments remains little explored [16], [17].

A four-component solution follows from the structure of the problem. The four stages differ in what is being decided, how quickly, and with what data (Table 2.2), so no single technique can serve all of them. At the same time the stages depend on one another, and their benefits can only be verified and sustained if they are connected. The decomposition is developed in Section 2.3.

2.3. Subproblem and component mapping

The overall problem divides along the lifecycle into four subproblems, SP1 to SP4. Each is assigned to one component (C1 to C4) and corresponds to one research question of the project (RQ1 to RQ4). Figure 2.1 shows the decomposition, and Table 2.2 shows why the subproblems do not overlap: each concerns a different decision, made on a different time scale, using different evidence. Component designs are summarised in Sections 3.3 to 3.6.

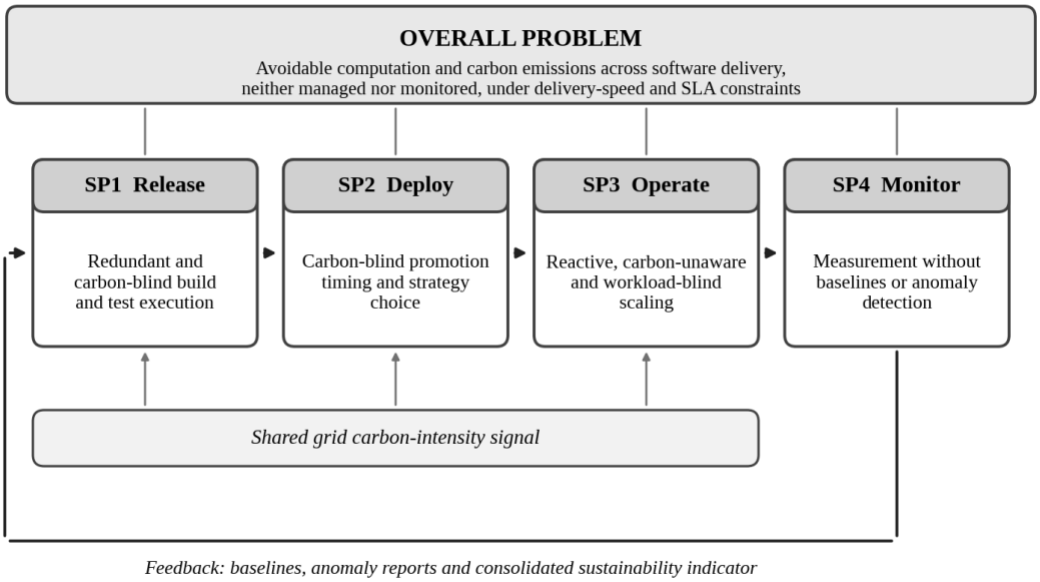


Figure 2.1 Decomposition of the overall problem into four lifecycle subproblems

Table 2.2 Distinguishing features of the four subproblems

Subproblem (component)	Decision concerned	Decision horizon	Principal evidence
SP1 Release (C1)	What to build and test for a commit, and when to start the pipeline	Per commit, with bounded delay	Change sets, build and test records, grid carbon intensity
SP2 Deploy (C2)	When and how to promote a build to production	Per release candidate, minutes to hours	Grid carbon intensity, system state, deployment history, per-phase energy and carbon
SP3 Operate (C3)	How many resources to run, and which jobs to run now or defer	Continuous, seconds to minutes	Runtime telemetry, forecast demand, carbon intensity, workload priority
SP4 Monitor (C4)	Whether environmental behaviour is normal or has regressed	Per pipeline execution and continuous	Stage-level history of duration, CPU, energy and carbon from SP1–SP3

2.3.1. Subproblem 1: redundant and carbon-blind build and test execution (Release)

At the Release stage a pipeline compiles and tests code for every commit. In multi-module repositories the usual configuration rebuilds and retests every module whatever the extent of the change, so the work done grows with the size of the repository and not with the size of the change [11], [12]. A documentation-only edit can therefore trigger the same full run as a change to a shared library. The pipeline also starts as soon as the commit arrives, without regard to grid carbon intensity [13], [14].

Test selection techniques address which tests are needed [11], [12], and carbon-aware scheduling addresses when flexible work should run [13], [14], but the two are not combined for pipeline execution. Both aspects must be handled together. Unnecessary work must be avoided, and

necessary work that is not urgent must be placed in a lower-carbon period within a bounded delay. Developers still need fast feedback, and selective builds must still catch failures (Table 1.1).

> **Subproblem 1.** CI/CD pipelines execute build and test work that a given change does not require, and execute the necessary work at times chosen without regard to grid carbon intensity, so energy use and emissions exceed what the change and its delivery deadline justify.

Associated research question (RQ1): To what extent can git-diff-based, module-level change-impact analysis, combined with a learned carbon-intensity classifier, reduce both unnecessary build and test execution and the carbon cost of necessary execution in CI/CD pipelines?

2.3.2. Subproblem 2: carbon-blind deployment timing and strategy (Deploy)

At the Deploy stage a build that has passed its checks is normally promoted to production straight away, so timing is set by the end of the build and not by grid conditions [13], [14]. The deployment strategy is chosen for risk and availability [22], although strategies differ in how much parallel infrastructure they need during a rollout. The energy and carbon of an individual deployment are not routinely visible to the team, so the choice cannot be informed by its own cost.

Existing work on sustainable deployment has concentrated on where deployment tasks are placed [21], and does not address when to release or which strategy to use under given grid conditions. Any remedy must also stay bounded. Postponing a release in the hope of a cleaner grid is acceptable only up to a limit that the delivery team accepts.

> **Subproblem 2.** Software is promoted to production when the build finishes, using a strategy chosen for reliability alone, without visibility of the deployment's own energy and carbon cost or of current grid conditions, so avoidable emissions are incurred and no principled bounded-delay alternative exists.

Associated research question (RQ2): How can real-time grid carbon-intensity signals and predictive analytics be incorporated into deployment scheduling to identify low-carbon deployment windows without degrading delivery performance?

2.3.3. Subproblem 3: reactive, carbon-unaware and workload-blind scaling (Operate)

Once deployed, services consume resources continuously. Kubernetes autoscaling is largely reactive: it adjusts capacity after CPU or memory utilisation crosses a threshold, so it lags demand, and it takes no account of the environmental cost of a scaling action [9], [15]. All workloads are treated alike, so delay-tolerant jobs run at times of high grid carbon intensity even when they could wait.

Predictive resource management can improve utilisation [15] and carbon-aware scheduling can shift flexible work [14], but the two are rarely joined with workload-priority-aware decisions in operational Kubernetes settings [9]. The difficulty has three parts. Scaling must anticipate demand, must weigh carbon, and must not breach SLAs for urgent work, so the system must know which jobs may be deferred.

> **Subproblem 3.** Kubernetes workloads are scaled reactively on performance metrics alone and all jobs are treated as equally urgent, so the environmental cost of scaling is ignored and delay-tolerant work cannot be moved away from high-carbon periods without a mechanism that protects SLA compliance.

Associated research question (RQ3): How accurately can an LSTM-based model predict Kubernetes workload demand, and how can carbon-emission estimation and workload prioritisation be integrated into scaling decisions to reduce environmental impact while preserving SLA compliance?

2.3.4. Subproblem 4: measurement without sustainability intelligence (Monitor)

Observability tools report performance, reliability and resource use, and standards such as SCI specify how emissions can be estimated [18]. Teams can therefore obtain numbers but little judgement. Sustainability metrics are reported without historical context, no baseline shows what is normal for a given pipeline stage, and a regression, such as a build that gradually becomes more energy-hungry, can pass unnoticed (Section 1.1). Stages differ in duration and resource profile, so behaviour that is abnormal in one stage may be normal in another.

Anomaly detection is a mature field [16], [17], but its use for sustainability monitoring in CI/CD environments is, to the best of the team's knowledge, little explored. Monitoring is also the only subproblem that can verify the other three. Without baselines and a consolidated indicator, it is not possible to tell whether release, deployment and scaling decisions are reducing environmental cost or increasing it, which is the early warning that managers need (Table 1.1).

> **Subproblem 4.** Energy and carbon measurements of delivery activity are collected, if at all, without stage-specific baselines, anomaly detection or a consolidated indicator, so regressions in environmental behaviour go undetected and the effect of the other three subproblems cannot be verified or fed back.

Associated research question (RQ4): How can sustainability monitoring be extended from measurement alone to historical baseline modelling and anomaly detection, enabling automatic identification of sustainability regressions across the Release, Deploy and Operate stages?

2.3.5. Interdependence and necessity of all four subproblems

The subproblems are non-overlapping in what they decide (Table 2.2) but interdependent in what they exchange. Table 2.3 sets out, at problem level, what each needs from the others, what it provides, and what remains unsolved if it is omitted. Interface details are defined in Chapter 4.

Table 2.3 Interdependence of the subproblems and consequence of omitting each

Subproblem	Needs	Provides	If left unsolved
SP1 Release	Shared carbon-intensity signal; feedback from monitoring on past behaviour	Build artefacts and build metadata with carbon context to SP2; execution records to SP4	Unnecessary, carbon-blind compute continues at the start of every delivery, before anything reaches production

Subproblem	Needs	Provides	If left unsolved
SP2 Deploy	Build outputs from SP1; carbon-intensity signal; deployment history	Deployment profiles and carbon estimates to SP3; deployment records to SP4	The step that places each change in production stays carbon-blind and its strategy overhead unexamined
SP3 Operate	Deployment profile from SP2; runtime telemetry; carbon-intensity signal; workload priorities	Scaling decisions and workload classifications; runtime records to SP4	Runtime resources stay reactive and uniform, and delay-tolerant work cannot be deferred
SP4 Monitor	Execution, deployment and runtime records from SP1–SP3	Baselines, anomaly reports and a consolidated indicator fed back to SP1–SP3	Gains cannot be verified, regressions go unseen and the feedback loop cannot be closed

All four are necessary for the same reason. Without SP1 the pipeline wastes energy before delivery begins. Without SP2 each change reaches production without regard to the grid. Without SP3 the running system continues to scale blindly. Without SP4 none of these improvements can be checked, and the loop that lets later decisions learn from earlier outcomes is missing. The overall problem is therefore a lifecycle problem, and addressing any subset leaves part of it open.

2.4. Overall aim and group objectives

> **Overall aim.** To design, implement and evaluate one integrated Green DevOps framework in which the Release, Deploy, Operate and Monitor stages share carbon-aware information through a closed feedback loop, so that unnecessary computation and avoidable carbon emissions across the software delivery lifecycle are reduced while delivery velocity, application performance and SLA compliance are preserved.

Six group objectives (GO1 to GO6) make the aim measurable. GO1 to GO4 correspond to the four subproblems and answer RQ1 to RQ4. GO5 concerns integration, and GO6 concerns evaluation of the framework as a whole. Table 2.4 maps each objective to the components involved, the evidence used to evaluate it and the section of this report where that evidence is presented.

Table 2.4 Group objectives, components and evaluation evidence

ID	Objective	Comp.	Evidence and measures	Section
GO1	Reduce unnecessary build and test execution, and the carbon cost of necessary execution, at the Release stage	C1	Paired optimised and unoptimised Jenkins runs on identical commits (wall-clock time, test-skip rate, analysis overhead); classifier performance against baselines under time-series cross-validation on hourly grid carbon-intensity data	6.3

ID	Objective	Comp.	Evidence and measures	Section
GO2	Measure deployment-stage energy and carbon, and select low-carbon deployment windows and strategies within bounded delay	C2	Deployment records, carbon reports and per-phase carbon snapshots across strategies; deploy or wait decisions with the carbon intensity at decision time and the window that followed	6.3
GO3	Forecast Kubernetes workload demand and integrate carbon estimation and SLA-aware prioritisation into scaling decisions	C3	Forecast accuracy (R^2, MAPE) on held-out Bitbrains fastStorage traces; scaling and delay recommendations produced by the decision engine; functional comparison with standard autoscaling	6.3
GO4	Establish stage-specific sustainability baselines, anomaly detection and a consolidated sustainability indicator	C4	Stage-level records (duration, CPU, energy, carbon, carbon intensity) from Jenkins pipeline executions; anomalies flagged by statistical and machine-learning detectors; indicator values	6.3
GO5	Integrate the four components with a shared carbon-intensity signal, defined interfaces and a feedback path from monitoring to earlier stages	C1–C4	Interface specification (Appendix A); integration and end-to-end tests of the complete pipeline	4.2, 6.2
GO6	Evaluate the aggregate reduction in unnecessary computation and carbon emissions while preserving delivery velocity, application performance and SLA compliance, including cases where environmental and cost or performance gains coincide	All	Consolidated results across components; overhead of the framework itself; stakeholder validation	6.1, 6.3–6.5

Each objective names the quantity to be observed, the baseline it is compared with and the data that supply it. Numerical acceptance criteria are fixed in Section 6.1 before results are presented, and the extent to which each objective was achieved is judged in Section 6.5.

2.5. Scope, assumptions and constraints

2.5.1. Scope and exclusions

The scope follows the evaluated prototype (Section 1.4). Table 2.5 states what is covered and what is not. Results should not be extended beyond these boundaries without further validation.

Table 2.5 Scope boundaries of the integrated solution

Dimension	Covered	Not covered

Dimension	Covered	Not covered
Build and CI	Maven-based multi-module projects in Jenkins pipelines	Other build systems and CI platforms
Runtime platform	A single K3s cluster on one virtual machine	Multi-cluster, multi-region and multi-cloud deployments
Electricity grid	Carbon-intensity data for one grid zone (Sri Lanka)	Other grid zones; moving work between regions (spatial shifting)
Workload data	Bitbrains fastStorage traces for demand forecasting	Live production traffic and other workload types
Emissions	Operational energy and carbon of pipeline, deployment and runtime activity, estimated from resource utilisation	Embodied (hardware manufacturing) emissions [3]; direct hardware power metering; claims about organisational, national or global emissions
Decisions	Bounded delay of flexible work; scaling and prioritisation of workloads	Optimisation of application source code; indefinite deferral of work

2.5.2. Assumptions

A1 **Carbon-intensity data are available and adequate.** The framework relies on an external grid carbon-intensity service. Intermittent outages are expected, so fail-safe behaviour is needed (Section 4.4).

A2 **Part of the workload is flexible.** Some builds, deployments and jobs can wait for a bounded period without harming users, and urgent work can be identified and allowed to proceed immediately [13], [14].

A3 **Energy can be estimated from resource utilisation.** Direct power measurement is usually impractical in virtualised environments, so energy is estimated from CPU and similar metrics [4], [25]. Absolute energy and carbon values are therefore estimates.

A4 **Historical data are representative.** Models trained on historical grid and workload data are assumed to generalise to the evaluated environment. Use elsewhere requires retraining and validation.

A5 **Service levels are defined.** Each workload has an SLA or urgency class, available or derivable, so that immediate and delay-tolerant work can be told apart.

2.5.3. Constraints

C1 **Delivery velocity.** Delays introduced by carbon-aware decisions must be bounded and must never be indefinite.

C2 **Service levels.** Scaling and deferral must not breach SLAs for immediate workloads.

C3 **Framework overhead.** The time and energy cost of the framework's own analysis and models must remain small relative to the work it optimises, and is measured (for example, the Release analysis adds about one second per pipeline execution, Section 1.3).

C4 **Evaluation environment and data.** The prototype was evaluated on a single virtual machine over the limited period of the project, with a modest number of pipeline executions and deployments. Results are therefore feasibility-level evidence and not statistically robust estimates of production impact (Section 6.6).

2.5.4. Dependencies

D1 **External carbon-intensity service.** All four components depend on a grid-signal source and on its availability and accuracy.

D2 **Host platforms.** The solution depends on Jenkins, Maven and Kubernetes (K3s), and on metrics collection and dashboard tooling for telemetry.

D3 **Inter-component dependencies.** Deploy depends on Release outputs, Operate on Deploy outputs and Monitor on all three, with feedback from Monitor to the earlier stages (Table 2.3). Interface agreements (Section 4.2) are therefore a prerequisite, and delay in one component can propagate to the others.

3. OVERALL SOLUTION STRUCTURE

3.1. Integrated solution overview

The solution is a single Green DevOps framework that treats the energy use and carbon intensity of software delivery as a decision input at every lifecycle stage. It is not four tools that each optimise one stage in isolation. It responds directly to the overall problem in Section 2.1 and to the three gaps in Section 2.2.3: lifecycle coverage (G1), shared signal and feedback (G2), and operational evidence (G3). Its four components are Release (C1), Deploy (C2), Operate (C3) and Monitor (C4). They correspond to subproblems SP1 to SP4 and are summarised in Sections 3.3 to 3.6.

Three design features make the components one system rather than a collection of parts.

A shared carbon context. All four components draw on the same live grid carbon-intensity signal for the Sri Lankan zone, obtained from Electricity Maps. Deterministic historical heuristics take over if the service is unavailable. Energy and emissions are accounted for using Software Carbon Intensity principles [18]. Decisions and measurements at different stages are therefore expressed in the same units and can be compared directly.

Chained outputs. Each stage passes its results to the next. Release hands build artefacts and metadata, with carbon context, to Deploy. Critically, the carbon context includes not only the current intensity reading but also the complete 168-hour rolling intensity history and a short-term forecast, which together form the feature vector that both scheduling engines consume. When Release defers a run to a greener window it also pre-selects a deployment strategy — computed by the Green AI agent at scheduling time — and carries it forward as a parameter so that Deploy inherits a decision that was made under the same carbon conditions. Deploy hands deployment profiles and carbon estimates, recorded at distinct phases (before, canary-live, and promoted or after), to Operate. All three stages send execution, deployment and runtime records to Monitor. Table 2.3 sets out these dependencies, and Chapter 4 defines the interfaces.

A closed feedback loop. Monitor builds stage-specific baselines from the records it receives, flags anomalies, and consolidates the results into a Sustainability Health Score from 0 to 100. These outputs feed back into later release, deployment and scaling decisions. This loop is what allows the framework to verify its own effect and to detect regressions, which none of the three optimising stages can do alone.

The framework applies the constraints set out in Section 2.5.3 throughout. Delivery speed and service levels are treated as limits on optimisation, not as costs to be traded away. Release can defer a non-urgent build by at most eight hours, and Deploy retries a deferred release every 30 minutes for at most six hours. Operate separates immediate, SLA-sensitive jobs from delayable ones, so only flexible work is ever moved. If a delay is not worthwhile, work proceeds at once.

Table 3.1 summarises how the stages work together as one framework.

Table 3.1 The integrated framework at a glance

Stage (component)	Question answered	Core mechanism	Output passed on

Release (C1)	What needs to be built and tested, and when should the pipeline run?	Git-diff module-level impact analysis; LightGBM carbon-intensity classifier for timing; Green AI agent pre-consultation for strategy; bounded-delay scheduling with pre-selected deploy strategy	Scoped build artefacts, carbon context (168-hour history and forecast), and pre-selected deploy strategy
Deploy (C2)	When and how should this build reach production?	Live greenCheck() confirmation of carbon conditions; pre-selected strategy honoured or overridden if conditions shift; canary health validation; carbon snapshots at before, canary-live, and promoted phases	Deployment profile and per-phase carbon estimate
Operate (C3)	How much capacity should run, and which jobs should run now?	LSTM workload forecasting, carbon estimation, SLA-aware job prioritisation in a runtime decision engine	Scaling recommendations and workload classifications
Monitor (C4)	Is environmental behaviour normal, or has it regressed?	Stage-specific baselines; Z-score and Isolation Forest anomaly detection; Sustainability Health Score	Baselines, anomaly reports and score fed back to C1 to C3

The central value of the solution is that a delivery team gets one carbon-aware capability across the whole lifecycle, without having to assemble and reconcile separate point solutions. It is also designed for low-effort adoption. The Release module runs as a single step in Jenkins or GitHub Actions, the Deploy logic is packaged as a Jenkins Shared Library, and Operate and Monitor run on the K3s cluster and report through Prometheus and Grafana dashboards.

The end-to-end outcome can be followed through one change. A commit enters the Release stage, where only the affected modules are built and tested. Before the build runs, the Build Optimizer invokes both the LightGBM carbon scheduler — which decides *when* to run — and the Green AI agent — which pre-selects *how* to deploy — in a single coordinated pre-flight phase. If the run is deferred to a greener window, the pre-selected strategy is carried forward so the Deploy stage inherits a decision made under the same carbon conditions. Once the build completes, Deploy performs a live greenCheck() immediately before promotion to confirm that conditions have not shifted; if they have, it logs the discrepancy but honours the original commitment. The resulting deployment is recorded at three carbon snapshots: before the deployment, with the canary instance live, and after the stable version is promoted. Once the application is live, Operate forecasts demand, scales proactively, and defers delay-tolerant jobs away from high-carbon periods. Throughout, Monitor records energy and carbon at each stage, compares them with baselines, and returns the findings to the earlier stages. Completed release runs are also persisted to a structured CSV telemetry log, which provides a per-build record of

optimizer decisions, scheduling outcomes, carbon readings, build and test durations, and the final deploy strategy for offline analysis. Rescheduled or aborted builds are excluded from this log so that only completed lifecycle records are counted.

In the evaluated prototype, this produced component-level evidence consistent with the overall aim, presented in Chapter 6:

- **Release:** a mean pipeline time reduction of 45.3% across 11 paired runs, with 91.3% of test executions skipped.
- **Deploy:** 17 deploy and 13 wait decisions, with 11 of the 13 waits followed by a greener window.
- **Operate:** an LSTM forecast R^2 of about 0.95 and a MAPE of 3.28%.
- **Monitor:** 63 stage-level records analysed.

These figures come from a single-VM, single-grid-zone prototype with small samples. They show that the integrated approach is feasible. They are not statistically robust estimates of production impact (Sections 2.5.3 and 6.6). Section 3.2 presents the high-level architecture of the framework (Figure 3.1).

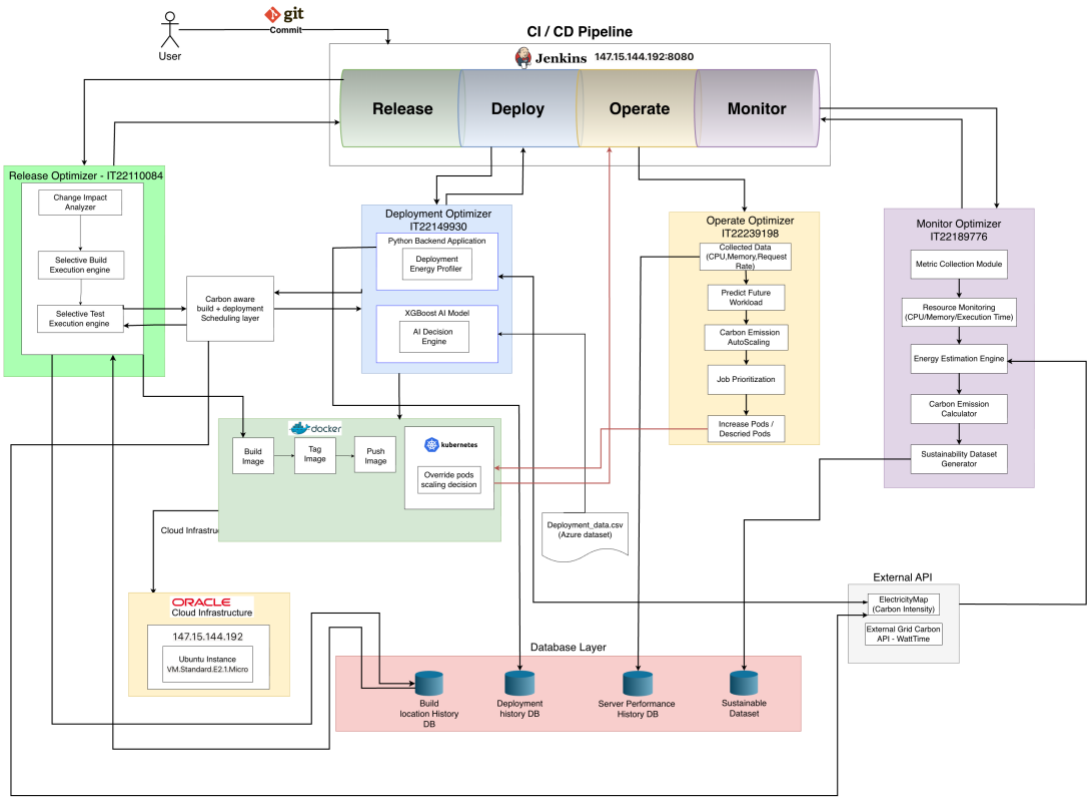


Figure 3.1 High level architecture

3.2. Component 1 summary

Component and owner. Release (C1), by Mendis J.D.L. (IT22110084). It addresses SP1 (redundant and carbon-blind build and test execution) and RQ1.

Inputs. The git diff between the base and head commits, the Maven module structure of the repository, hourly grid carbon-intensity history for the Sri Lankan zone, and the urgency of the run.

Core method. Changed files are mapped to their owning Maven modules by longest-prefix path matching, and the affected set is expanded using Maven's also-make (`-am`) reactor semantics. Documentation-only changes skip execution, while changes to shared modules or build configuration files trigger broader validation. A LightGBM classifier then judges whether the current grid hour is a low-carbon "green window". If it is not, a Prophet-based forecaster looks for a lower-carbon hour within a maximum delay of eight hours. If none is found, the build runs immediately, so developer feedback is never held back indefinitely.

Outputs. A scoped build and test plan, a run-now or delayed decision, and build artefacts with metadata and carbon context for C2. Execution records go to C4.

Contribution to the integrated solution. C1 removes unnecessary compute at the start of the lifecycle and sets the carbon context that later stages inherit.

Evidence summary. The classifier reached a macro F1-score of about 0.98 under time-series cross-validation, ahead of all baselines. Across 11 paired Jenkins runs, mean wall-clock time fell by 45.3% and 91.3% of test executions were skipped (13 of 149), at an analysis overhead of about one second per run. The gains were uneven, and two single-module runs were slightly slower. The Prophet forecaster was the weakest result, with a MAPE of about 51%, which is why the immediate-run fallback is essential. Detailed evidence is in the Component 1 thesis.

3.3. Component 2 summary

Component and owner. Deploy (C2), by Rathnayake D.M.B.H. (IT22149930). It addresses SP2 (carbon-blind deployment timing and strategy) and RQ2.

Inputs. Build outputs and metadata from C1, live grid carbon intensity, system resource state, the time of day, and the history of earlier deployments.

Core method. A ReAct-based AI agent decides when and how to promote a build. It calls five tools (carbon intensity, system resources, time context, deployment history and green-window finder) and passes the results to a locally hosted language model, which reasons over the evidence and returns a structured decision. A six-factor weighted Green Score from 0 to 100 is included in the prompt to anchor the reasoning in measured data. The agent then chooses recreate (1.0× infrastructure overhead), rolling (1.1×) or canary (1.2×), or waits. A deferred release is

retried every 30 minutes for up to six hours, and a rule-based fallback applies if the model is unavailable. The logic is packaged as a Jenkins Shared Library.

Outputs. A deploy or wait decision with a chosen strategy and an explainable decision score. A deployment profile and carbon estimate go to C3, and deployment records, carbon reports and per-phase carbon snapshots go to C4.

Contribution to the integrated solution. C2 makes the promotion step carbon-aware, makes the energy cost of each strategy visible, and links the build output from C1 to the running system managed by C3.

Evidence summary. The evaluation covered 35 deployment records and 30 agent decisions on one server. The agent made 17 deploy and 13 wait decisions, and 11 of the 13 waits were followed by a greener window. The language model handled 93.3% of decisions, and the mean decision score was 78.2 out of 100. Mean emissions were about 0.74 gCO_2 for recreate, 0.76 gCO_2 for rolling and 1.27 gCO_2 for canary. Samples per strategy were small, and 70% of decisions were made under high-carbon conditions. Detailed evidence is in the Component 2 thesis.

3.4. Component 3 summary

Component and owner. Operate (C3), by Thassara M.P.M. (IT22281432). It addresses SP3 (reactive, carbon-unaware and workload-blind scaling) and RQ3.

Inputs. Kubernetes runtime telemetry (CPU, memory, request rate, active pods and latency), the deployment profile from C2, live carbon-intensity estimates, and the urgency or SLA class of each workload.

Core method. An LSTM model forecasts near-term resource demand over 30-second horizons, so scaling can anticipate load instead of reacting to it. A Carbon Estimation Engine assesses the projected energy and emissions of candidate scaling actions. A Job Priority Engine classifies workloads as immediate (SLA-sensitive) or delayable. A Runtime Decision Engine combines the three and recommends one of Scale Up, Scale Down, Maintain or Delay Jobs. Only delayable work is ever deferred, which protects SLA compliance.

Outputs. A scaling recommendation, workload classifications and carbon-aware operational decisions. Runtime records go to C4.

Contribution to the integrated solution. C3 extends carbon-awareness from delivery into continuous runtime operation, where resources are consumed for the longest period. It completes the path from commit to running service.

Evidence summary. On held-out Bitbrains fastStorage traces, the forecaster achieved an R^2 of about 0.95 and a MAPE of 3.28%. The decision engine produced all four recommendation types, and a functional comparison with the standard Horizontal Pod Autoscaler showed capabilities that the standard autoscaler lacks (forecasting, carbon awareness, prioritisation and SLA-aware

decisions). The evidence is forecast accuracy and functional feasibility on offline traces, not a measured emissions reduction in a live cluster. Detailed evidence is in the Component 3 thesis.

3.5. Component 4 summary

Component and owner. Monitor (C4), by Kodithuwakku I.P. (IT22189776). It addresses SP4 (measurement without sustainability intelligence) and RQ4.

Inputs. Stage-level records for Build, Test and Deploy activity (duration, CPU utilisation and infrastructure overhead) from Jenkins pipeline executions, plus live regional carbon intensity.

Core method. Because direct hardware power measurement is impractical, power is estimated from average CPU utilisation by linear interpolation between a reference idle power of 10 W and a maximum of 80 W [25]. Energy follows from the measured workload duration, and emissions are obtained by multiplying energy by carbon intensity [18]. Stage-specific historical baselines are built from these records. A Z-score layer flags substantial deviations transparently, and unsupervised Isolation Forest models, trained separately for each stage, flag multidimensional anomalies without labelled data [16], [17]. The results are consolidated into a Sustainability Health Score from 0 to 100 and shown on Prometheus and Grafana dashboards.

Outputs. Baselines, anomaly reports and the Sustainability Health Score, fed back to C1 to C3.

Contribution to the integrated solution. C4 closes the feedback loop. It is the only component that can verify whether the other three are reducing environmental cost, and it detects regressions that would otherwise go unnoticed.

Evidence summary. The evaluation used 21 Jenkins executions, giving 63 stage-level records. The Z-score layer flagged eight anomalies (four Build, three Test and one Deploy), and the Isolation Forest flagged six. The recorded totals of 0.001163 kWh and 0.000502 $kgCO_2eq$ are proof-of-concept measurements of the monitoring pipeline itself. Without labelled ground truth, detection accuracy was not measured, and the validity of the Sustainability Health Score as a measure of sustainability has not been tested independently. Detailed evidence is in the Component 4 thesis.

3.6. End to end user and data flow

This section follows one change through the whole framework, from a developer's commit to a running service and back to the monitoring dashboards. It shows where a user acts, what data moves between components, and where each of the four components interacts with the others. Figure 3.2 shows the flow, and Table 3.2 lists the data items that move through it. Interface details are specified in Section 4.2 and Appendix A, and failure paths are covered in Section 4.4.

Users in the flow

Four groups of people touch the framework, and each interacts at a different point (Table 3.1 in Section 1.2 lists their needs).

- **Developers** commit code, which starts the flow, and receive the build and test results.
- **DevOps and platform engineers** set up the pipeline step and library, configure the framework, and handle deferred or failed runs.
- **Site reliability and operations engineers** use the scaling recommendations and the dashboards for the running system.
- **Engineering managers and sustainability officers** read the Sustainability Health Score and anomaly reports.

Apart from the commit and the initial configuration, no user action is needed for the flow to proceed.

Walk-through of one change

Step 1: commit and trigger. A developer commits to a Maven-based multi-module repository, and the Jenkins pipeline starts. The Release module receives the git diff between the base and head commits.

Step 2: change-impact analysis (Release). Changed files are mapped to their owning modules, and the affected set is expanded with Maven's also-make semantics. A documentation-only change skips execution, and a change to a shared module or build configuration triggers wider validation. The output is a scoped build and test plan.

Step 3: pre-flight carbon decision (Release). Before the build runs, the Build Optimizer makes one coordinated pre-flight call to two models. The LightGBM carbon scheduler decides when to run, using the carbon context, which consists of the current reading, the 168-hour rolling intensity history and a short-term forecast. The Green AI agent decides how to deploy and pre-selects a deployment strategy. If the grid is classified as high-carbon and a lower-carbon hour exists within the eight-hour limit, a non-urgent build is deferred. Otherwise it runs immediately, so a developer is never held back indefinitely.

Step 4: build and test (Release). The scoped build and test run. A completed run is logged to the CSV telemetry log, and rescheduled or aborted builds are left out. The build artefacts, the carbon context and the pre-selected strategy pass to Deploy, and the execution record goes to Monitor.

Step 5: deployment decision (Deploy). Deploy receives the Release outputs. Immediately before promotion, a live carbon check confirms the current conditions. If they have shifted, the discrepancy is logged and the original commitment is honoured. If conditions are unfavourable and no commitment applies, the agent waits and retries every 30 minutes for up to six hours. The agent calls its five tools, produces a deploy or wait decision with a Green Score and an explainable decision score, and selects recreate, rolling or canary.

Step 6: deployment and carbon snapshots (Deploy). The build is deployed with the chosen strategy, with canary health validation where it applies. Carbon snapshots are recorded at three phases (before, canary-live and promoted). The deployment profile and snapshots pass to Operate, and the deployment record, carbon report and snapshots pass to Monitor.

Step 7: runtime operation (Operate). Once the application is running, the LSTM forecasts near-term demand from Kubernetes telemetry. The Job Priority Engine classifies workloads as immediate (SLA-sensitive) or delayable. The Carbon Estimation Engine assesses the carbon impact of candidate actions. The Runtime Decision Engine combines these and recommends Scale Up, Scale Down, Maintain or Delay Jobs. Only delayable work is deferred, which protects SLAs. Runtime records go to Monitor.

Step 8: monitoring (Monitor). Monitor collects execution, deployment and runtime records from the three stages. It estimates energy and carbon from CPU utilisation and carbon intensity, compares each record with the baseline for its stage, flags anomalies with the Z-score layer and the stage-specific Isolation Forest, and updates the Sustainability Health Score. Results appear on the Prometheus and Grafana dashboards.

Step 9: feedback. Baselines, anomaly reports and the Sustainability Health Score are fed back to Release, Deploy and Operate, so that later build scheduling, deployment timing, scaling policy and workload prioritisation draw on the history of earlier outcomes.

Data items in the flow

Table 3.2 Data items moving through the framework

Data item	Produced by	Consumed by	Content	Interface
Git diff and module map	Git, Maven	C1	Changed files and their owning modules	Internal to C1
Carbon context	External service, via C1	C1 (scheduler and agent), C2	Current intensity, 168-hour history, short-term forecast	I1, I2
Build and test plan, execution record	C1	C2 (artefacts), C4 (record)	Scoped modules, durations, skip counts	I2, I4
Pre-selected deployment strategy	C1 (Green AI agent)	C2	Strategy chosen under the scheduling-time carbon conditions	I2
Deploy or wait decision	C2	Jenkins pipeline	Decision, strategy, Green Score, decision score	Internal to C2
Deployment profile and carbon snapshots	C2	C3, C4	Strategy, overhead and carbon at three phases	I3, I4
Runtime telemetry	Kubernetes, Prometheus	C3, C4	CPU, memory, request rate, pods, latency	Prometheus
Workload forecast and classification	C3	C3 decision engine	Predicted demand; immediate or delayable	Internal to C3
Scaling recommendation	C3	Cluster, operators	Scale Up, Scale Down, Maintain or Delay Jobs	Cluster interface
Stage-level	C1, C2, C3	C4	Duration, CPU,	I4

sustainability records			overhead, energy, carbon, intensity, status	
Baselines, anomaly reports, health score	C4	C1, C2, C3, dashboards	Per-stage baselines, flags, 0 to 100 score	I5, Grafana

Where the components interact

The flow has three types of interaction. A forward chain carries build outputs and carbon context from Release to Deploy and the deployment profile from Deploy to Operate. A collection path carries records from all three stages to Monitor. A feedback path carries Monitor's findings back to the earlier stages. Release and Deploy interact most closely, because the pre-flight consultation lets the deployment strategy be chosen at scheduling time and inherited later. The external carbon service touches all four components, which is why a fallback to historical heuristics is shared by all of them. The timing of the stages differs. Release and Deploy act once per commit or release, Operate acts continuously over seconds to minutes, and Monitor acts on every record and over longer periods (Table 2.2).

4. COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1. Integration strategy

The four components were integrated as loosely coupled modules that exchange small structured records, not as one codebase in which each stage calls the internals of another. Each component can run and be tested on its own, and the integration consists of agreed hand-offs between them. This follows the dependency structure in Section 2.5.4 (D3): Deploy depends on Release, Operate on Deploy, and Monitor on all three. It also follows Section 2.5.4's point that interface agreements are a prerequisite, because a delay or a change in one component can propagate to the others.

Integration backbone. The two delivery stages are joined through the Jenkins pipeline. Release runs as a self-contained Python package invoked from a single pipeline step, which avoids inter-service network calls. Deploy is a Jenkins Shared Library invoked with a single function call. Release passes its results to Deploy as pipeline parameters. The runtime stages are joined through the K3s cluster and its telemetry. Operate reads Kubernetes metrics, and Monitor collects records from all three stages and exposes them through Prometheus and Grafana dashboards.

The tightest coupling: Release and Deploy. Before a build runs, the Build Optimizer invokes the LightGBM carbon scheduler (when to run) and the Green AI agent (how to deploy) in one coordinated pre-flight phase. Both consume the same carbon context, which is the current reading, the 168-hour rolling intensity history and a short-term forecast. If the run is deferred to a greener window, the pre-selected deployment strategy travels with it. Deploy therefore inherits a decision made under the same carbon conditions. Immediately before promotion, Deploy performs a live carbon check to confirm that conditions have not shifted. If they have, it logs the discrepancy and honours the original commitment, so a deferred release is not delayed a second time by the same signal. Deploy also records carbon snapshots at three phases (before deployment, with the canary live, and after promotion), which become the deployment profile passed to Operate and Monitor.

Sequence. Integration followed the direction of data flow, so that each hand-off was verified before the next stage was connected to it (Table 4.1).

Table 4.1 Integration sequence, interfaces and ownership

Step	Interface	Producer → consumer	What is exchanged	Interface owner	Verified by
I1	Shared carbon signal	External service → C1 to C4	Live intensity for the Sri Lankan zone, with a historical-heuristic fallback		Behaviour of each component with the live signal and with the service unavailable

I2	Release → Deploy	C1 → C2	Scoped build artefacts, carbon context (history and forecast), run-now or delayed decision, pre-selected strategy		A deferred run reaching Deploy with its strategy intact; live carbon check at promotion
I3	Deploy → Operate	C2 → C3	Deployment profile and per-phase carbon snapshots		Operate decisions generated from a real deployment profile
I4	Records to Monitor	C1 to C3 → C4	Execution, deployment and runtime records; completed release runs from the CSV telemetry log		Stage-level records appearing in the Monitor repository and dashboards
I5	Feedback	C4 → C1 to C3	Baselines, anomaly reports, Sustainability Health Score		

The interface owner is the developer of the producing component, who defines and documents the contract (Appendix A). The consuming developer validates inputs against it. A change to an interface is agreed between the two before it is made, and recorded in Appendix A and in version control.

Environments. Components were developed individually and then integrated on one Oracle Cloud virtual machine running Ubuntu, with K3s, Jenkins, a locally hosted language model for the Deploy agent, and Prometheus and Grafana. The same machine hosts the integrated evaluation described in Chapter 6. This keeps the integrated environment close to the evaluation environment, but it also limits the evidence to a single-VM, single-cluster setting (Section 2.5.1).

Managing integration risks. The main integration risks were identified from the dependencies in Table 2.3 and managed as shown in Table 4.2.

Table 4.2 Integration risks and how they were managed

Risk	Where it arises	Management measure
Carbon service outage or poor data	All components (D1)	One shared signal with deterministic historical fallback, so no component blocks on the service (Section 4.4)
Language-model unavailable or unreliable	Deploy and the Release pre-flight	Rule-based fallback decision (used in 6.7% of evaluated decisions)
Carbon conditions change between a decision and its execution	Release → Deploy	Live check at promotion; discrepancy logged and the original commitment honoured

Delays from different stages add up	Release and Deploy	Each delay is bounded (eight hours at Release; retries every 30 minutes for up to six hours at Deploy); urgent work proceeds immediately (Section 4.4)
Interface drift between developers	All hand-offs	Producer-owned, documented contracts (Appendix A), agreed before change
Skewed or duplicated records	Records to Monitor	Rescheduled and aborted builds excluded from the telemetry log, so only completed runs are counted
Framework overhead cancels the saving	Release	Analysis overhead measured (about one second per run) and reported (Section 6.3)

Section 4.2 defines the contracts behind these interfaces, Section 4.4 describes the failure paths in detail, and Section 6.2 reports the integration tests.

4.2. Interfaces data contracts and shared models

4.3. Shared nonfunctional requirements

The four components share a set of nonfunctional requirements (NFRs). They are defined once at framework level, not per component, because a quality that holds in only one stage does not hold for the pipeline as a whole. A fast Release stage does not help if the Deploy stage stalls on a failed external service, and a transparent Monitor stage does not help if the decisions feeding it cannot be explained. The requirements are organised using the quality characteristics of ISO/IEC 25010 , and the first three follow directly from the project constraints in Section 2.5.3: bounded delay (C1), SLA protection (C2) and low framework overhead (C3). Table 4.3 states each requirement, how the framework meets it, and whether the project measured it or only designed for it.

Table 4.3 Shared nonfunctional requirements

Requirement	Shared requirement	How the framework meets it	Status
Performance and overhead (C3)	The time and energy cost of the framework's own analysis must stay small relative to the work it optimises	Lightweight per-run analysis at Release (about one second, roughly 1% of optimised pipeline time); short-horizon forecasting and rule-based scoring at Operate and Deploy	Measured for Release;
Delivery velocity (C1)	Carbon-aware delay must be bounded and never indefinite	Release defers at most eight hours; Deploy retries every 30 minutes for at most six hours; work proceeds at once if no greener window is found	Designed; behaviour observed in the Deploy evaluation (wait decisions followed by a

			greener window)
Service-level compliance (C2)	Scaling and deferral must not breach SLAs for immediate work	Operate separates immediate from delayable jobs, and only delayable jobs are deferred	Designed; functional evidence only (Section 6.3)
Reliability and fault tolerance	A failure of any dependency must degrade the framework to normal, non-carbon-aware behaviour, never block delivery	Historical-heuristic fallback when the carbon service is unavailable; rule-based fallback when the language model fails (6.7% of evaluated decisions); immediate execution if no greener window exists; canary health validation before promotion	Fallbacks exercised in evaluation;
Maintainability and reproducibility	Components must be modular, configurable and independently testable	Release is a configuration-driven Python package; Deploy is a Jenkins Shared Library; stage-specific models at Monitor; code, trained models, scripts and logs published in the project repository (release v1.0-submission)	Designed; reproducibility artefacts published
Scalability	Behaviour must remain workable as repositories, pipelines and clusters grow	Release cost depends on change size rather than repository size; baselines are kept per stage	Not tested beyond one VM and one K3s cluster (Section 2.5.1)
Security	Credentials and pipeline data must be protected	Carbon-service credentials held in the CI secrets store; the language model is hosted locally, so pipeline data is not sent to external AI services; the only external call is for carbon intensity	Designed;
Privacy	The framework must not collect personal data it does not need	Inputs are operational: changed file paths, resource utilisation, durations and grid readings;	Designed; to be covered in Section 7.4
Usability and adoption	Setup effort must be low and decisions must be understandable	One pipeline step or function call per component; configuration file for Release; Grafana dashboards showing the 0–100 Sustainability Health Score; explainable decision score (mean 78.2 out of 100)	Dashboards and scores implemented;

Transparency and explainability	Every automated decision must be traceable	Z-score detections are interpretable; the Deploy agent returns a structured decision with a Green Score; SHAP attribution was applied to the classifier; completed release runs are logged to a telemetry file	Implemented
Accessibility	Interfaces should not exclude users with different needs	The user-facing surface is Grafana dashboards, which provide their own accessibility features;	No formal accessibility evaluation

Sustainability of the framework itself. The framework must not cost more energy than it saves. The Release analysis overhead is measured and small. However, the energy used by the locally hosted language model during Deploy decisions, by LSTM inference and by the monitoring stack has not been measured. This should be acknowledged as a limitation in Section 6.6 and not assumed to be negligible.

Trade-offs between requirements. Three tensions were resolved explicitly. First, delivery velocity takes priority over carbon savings, so every delay has a hard bound, and the framework gives up some emissions reduction to protect delivery speed. Second, reliability takes priority over decision quality, so a rule-based fallback is accepted even though it is less adaptive than the language model. Third, explainability takes priority over raw detection power at Monitor, which is why the interpretable Z-score layer is kept alongside the Isolation Forest.

Professional standards. Energy and emissions accounting follows the Software Carbon Intensity specification [18]. Quality characteristics follow ISO/IEC 25010. Data protection and ethical obligations are discussed in Section 7.4.

4.4. Integrated workflow and exception handling

This section describes how the framework runs end to end as implemented in the integrated pipeline, and what happens when a dependency or a stage fails. It is based on the implementation details reported by the owners of the Release and Monitor components, on the Deploy owner's component record, and on the pipeline definition. The Operate component is marked as pending where its details are not yet available.

4.4.1. Normal end-to-end workflow

The Release and Deploy stages run in one Jenkins pipeline that uses the `green-agent` Shared Library, and Jenkins allows only one build at a time. A normal run proceeds as follows.

1. **Trigger and initialisation.** A GitHub push starts the pipeline. All environment variables take deterministic defaults (for example, the deployment strategy defaults to rolling). If

the build is a rescheduled one, the pre-selected strategy passed to it is applied immediately.

2. **Checkout.** The source is fetched, and the commit message is inspected for the `[urgent]` and `[bypass-green]` flags. The previous successfully built commit is read and used as the diff base.
3. **Release analysis.** A release monitoring session is opened. The optimiser, running in a Docker container, discovers the Maven modules, builds a reverse dependency graph and determines the transitively affected modules. It then runs the carbon scheduler. The scheduler loads a local 168-hour intensity history, refreshes it from Electricity Maps if it is more than two hours old, fetches the current intensity and a 24-hour forecast, and applies the LightGBM classifier.
4. **Green scheduling.** Unless the commit is urgent or bypassed, the pipeline calls the Green AI agent over HTTP through the Shared Library. The combined decision gives a green probability, a confidence and a pre-selected deployment strategy. If a delay is recommended, a new build is queued with green scheduling disabled and the pre-selected strategy attached, and the current build ends as rescheduled. Otherwise the pipeline continues at once with the pre-selected strategy.
5. **Selective build and test.** Only the affected modules are built and tested, and test counts are read from the Surefire reports. If no work is planned, the monitoring session is skipped with a reason.
6. **Image build.** A Docker image is built, but only if the application module is affected. The release monitoring session is then closed.
7. **Strategy confirmation.** A single live carbon check asks the agent whether conditions have changed since scheduling. If the strategy differs, the discrepancy is logged and the pre-selected strategy is kept.
8. **Deployment.** The build is deployed using the selected strategy (recreate, rolling or canary), with carbon snapshots recorded and, for canary, a health check and an observation window.
9. Monitoring. For release and deploy, Jenkins calls the monitor command line to start and stop a sampling session. On stop, Monitor reads the energy counters, fetches the grid carbon intensity, compares the record with historical baselines, applies the Isolation Forest models and the rule-based health score, and stores a lifecycle record in MongoDB. A Flask dashboard displays the stored data. For the Operate stage, Monitor reads container CPU metrics from Prometheus.
10. **Release telemetry.** At the end of a completed build, one row is appended to the release metrics CSV. Rescheduled and aborted builds are not written.

Table 4.4 shows how commit flags and parameters change this flow.

Table 4.4 Controls that alter the normal workflow

Control	Effect
`[urgent]` in the commit message	Skips all green checks, including the strategy confirmation
`[bypass-green]` in the	Skips the scheduling delay only; the strategy confirmation still

commit message	runs
Force full build parameter	Skips the optimiser analysis and builds and tests all modules; green scheduling is also skipped
Dry run parameter	Runs the optimiser only; no build, test, image, deployment or monitoring session
Pre-selected strategy parameter	Carries the strategy from a rescheduled build into the new one
Green scheduling disabled	Set on a rescheduled build, so it always proceeds whatever the grid conditions

4.4.2. Failure paths and fallbacks

Table 4.5 consolidates the failure paths by dependency. Each component handles its own dependencies and no central coordinator exists, so the table names the component for each row.

Table 4.5 Failure paths, fallbacks and effect on delivery

ID	Scenario	Component	Response	Effect on pipeline	Status
F1	Carbon service unavailable or times out	Release	15-second timeout; uses the last known history value, or 300 gCO_2eq/kWh if there is none; forecast left empty	None; decision made on a stale signal	Implemented
F2	Carbon service unavailable	Deploy	Retries with exponential backoff, uses a five-minute cache, then a time-of-day heuristic	None	Implemented
F3	Carbon service unavailable, missing key or malformed reply	Monitor	10-second timeout; constant 0.5 kg/kWh (500 gCO_2/kWh); no retry; stale data is not detected	None; record uses the default	Implemented; staleness not handled
F4	Less than 168 hours of history available (free tier returns 24 hours)	Release	Backfills from the service; pads older hours with synthetic readings	Lower prediction confidence	Implemented
F5	LightGBM model missing, dependency	Release	Falls back to the rule-based	Decision still produced	Implemented

	missing, feature error or fewer than three readings		scheduling engine		
F6	Green AI agent unreachable, malformed output or invalid strategy	Release	Keeps the current or pre-selected strategy (default rolling) and logs a warning	No delay	Implemented
F7	Language model unavailable	Deploy	Score-based deterministic fallback, recorded in the decision log	None	Implemented
F8	Conditions change between scheduling and deployment	Release, Deploy	Pre-selected strategy honoured; discrepancy logged	None	Implemented
F9	Documentation-only change, no changes or empty diff	Release	Optimiser reports no changes; later stages skipped; monitoring skipped with a reason; telemetry row written	Build avoided	Implemented
F10	First commit, no diff base	Release	Falls back to the parent commit; behaviour on a repository's very first commit untested	Possible no-changes result	Partial
F11	Optimiser container, Maven build or test fails	Release	Pipeline fails; failure notification sent; nothing passed downstream	Delivery stops	Implemented
F12	Canary health check fails (12 attempts, 5 s apart) or more than five errors in the 60-second window	Deploy stage	Pipeline fails and the canary container is removed	Release halted	Implemented
F13	Monitor service unavailable, errors or exceeds the one-minute command timeout	Release	Exception caught; warning logged; pipeline continues	No lifecycle record for the run	Implemented
F14	Build aborted or rescheduled	Release	Open monitoring sessions	No stale session	Implemented

			cancelled; no telemetry row		
F15	Jenkins stops before the stop command (hard abort)	Monitor	No sampler timeout or orphan cleanup	Sampling process may keep running	Not handled
F16	Fewer than 10 historical records	Monitor	Isolation Forest skipped; status "Warming Up"; static health deductions; recovers once records accumulate	None	Implemented
F17	Isolation Forest training or prediction raises an exception	Monitor	Exception propagates; no lifecycle record produced	Pipeline continues; record lost	Not handled
F18	MongoDB unavailable (3-second selection timeout)	Monitor	Save returns false; record persisted to a local CSV; failure logged	None	Implemented
F19	Prometheus unavailable or malformed (3-second timeout)	Monitor	Returns 502 or 503; dashboard shows CPU as unavailable	None	Implemented
F20	Build, test or deploy stage fails	Monitor	Failure recorded; 20-point health score deduction; no retry or repair	Monitor is observational only	Implemented
F21	Release telemetry CSV write fails	Release	Warning logged; result unchanged	None	Implemented
F22	Operate failures (telemetry gaps, forecaster unavailable, no history, scaling failure, deferred-job limits)	Operate			

4.4.3. Fallback hierarchy

The design principle is that a failure in a carbon-aware part should reduce the framework to ordinary, non-carbon-aware behaviour and should not stop delivery. Three degradation chains implement this.

- **Carbon signal.** Live reading, then last known or cached value, then synthetic or heuristic estimate, then a fixed constant.
- **Scheduling decision.** LightGBM classifier, then the rule-based engine, then execution with the default rolling strategy.
- **Strategy decision.** Language-model agent, then score-based fallback, then keeping the pre-selected or default strategy.
- **Persistence and analysis in Monitor.** MongoDB, then local CSV; Isolation Forest, then static health-score deductions during warm-up.

Failures that do stop delivery are those in the delivery work itself: optimiser, build, test and canary failures (F11, F12). Failures in Monitor never block the pipeline, but they can lose the record of a run (F13, F17).

4.4.4. Bounds and guarantees

Table 4.6 Limits that keep the framework from blocking delivery

Limit	Value	Component	When reached
Concurrent builds	One	Pipeline	Further triggers queue
Delay by green scheduling	One deferral per change; a rescheduled build never defers again; target hour is chosen within the 24-hour forecast window	Release	Rescheduled build runs at once
Strategy confirmation	Single live check; no waiting	Release, Deploy	Pre-selected strategy honoured
Carbon service timeout	15 s (Release); 10 s (Monitor)	Release, Monitor	Fallback value used (F1, F3)
Monitor command timeout	1 minute	Release	Warning; pipeline continues
Prometheus and MongoDB timeouts	3 s each	Monitor	502 or 503; CSV fallback
Canary health and observation	12 × 5 s; 60 s window; 5 errors	Deploy stage	Pipeline fails
Rolling deployment	15 s between replacements; no health-check retry	Deploy stage	Failure on a slow start
Minimum data for models	3 readings (Release); 10 records (Monitor)	Release, Monitor	Rule-based engine; warm-up status
History retained and staleness threshold	168 hours; 2 hours	Release	Oldest entries trimmed; backfill

Delay bound. In the integrated pipeline, a change is deferred at most once. The deferred build has green scheduling disabled and the strategy confirmation does not wait, so there is no cumulative delay across stages. The eight-hour limit stated in earlier chapters was not found in the Release code, where the target hour is the lowest-intensity hour within the 24-hour forecast window. The Deploy retry loop (every 30 minutes for up to six hours) does not appear in the integrated pipeline, which calls the agent in single-shot mode.

Residual risk to the delay bound. No timeout is set on the calls to the Green AI agent, so if the agent hangs, the build hangs. This is the one situation in which the guarantee that delay is never indefinite (constraint C1) does not hold.

4.4.5. Recovery and logging

Recovery is mostly by fallback and not by retry. Release and Monitor make a single attempt at each external call, and no component retries with backoff except the Deploy carbon client. The exceptions are Monitor's warm-up, which recovers by itself as records accumulate, MongoDB, whose records are redirected to CSV, and the rescheduled build, which is the recovery path for a deferral. Failures are recorded in the Jenkins console, in the description of a rescheduled build, in the release metrics file, in MongoDB (or the CSV fallback), and in the agent's decision log, which keeps the latest 500 entries.

4.4.6. Cross-component behaviour and the feedback path

Release sends monitoring commands to Monitor and does not depend on the result. Monitor's exit codes are ignored by Jenkins, so a monitoring failure never blocks a release. Deploy receives the strategy and carbon context from Release as pipeline variables in the same Jenkinsfile, and Operate and Monitor consume data through Prometheus and the monitor command.

The designed feedback path, in which Monitor's health scores and anomaly reports would influence later release, deployment and scaling decisions, is **not implemented**. Monitor identifies anomalies and low scores, but no abort or adjustment signal is sent back to Release, Deploy or Operate. Monitor currently acts as an analytical observer, and the closed loop described in the architecture is design only. Section 8.3 lists it as future work.

Each component also has its own carbon-intensity client, and their fallbacks differ in value, unit and method (F1 to F3). The framework shares a data source but does not yet share a single client or fallback.

4.4.7. Residual gaps and risks

- No retry or backoff for the carbon service in Release and Monitor, and no staleness check on carbon data.
- Hours 25 to 168 of the Release history are synthetic when only a free-tier key is available, which reduces the accuracy of long-lag features.
- No timeout on Green AI agent calls (Section 4.4.4).

- No limit on the number of reschedules, which is prevented only by the disabled-scheduling flag on the rescheduled build.
- Orphaned Monitor sampler after a hard abort, and no maximum sampler duration.
- No exception handling around Isolation Forest training, and the model is retrained on each analysis, so training time grows with history and could approach the one-minute monitor command timeout.
- Release metrics are appended to a CSV with no locking.
- Rolling deployments have no health-check retry.
- The exception paths have no automated tests and were verified manually in pipeline logs.
- The first-commit case is untested, and the carbon scheduler is fixed to one grid zone.
- Operate's failure handling is not yet documented.

4.5. Team contribution and dependency matrix

The project was divided along the delivery lifecycle. Each member owns one component from research through development and evaluation, and writes the matching individual thesis. Integration work is shared, and each task has a named lead. This section records who did what and how the components depend on one another. Appendix B holds the contribution declaration confirmed by all four members, and Section 5.4 summarises the supporting evidence.

4.5.1. Individual contributions

Table 4.7 maps each member to a component. The subproblems do not overlap in decision, time scale or evidence (Table 2.2), so the division of work is deliberate. No two members worked on the same decision problem.

Table 4.7 Individual contributions by component

Member (ID)	Component	Research tasks	Development tasks	Test and evaluation evidence	Individual thesis
Mendis J.D.L. (IT22110084)	C1 Release (SP1)	Regression test selection, incremental builds, carbon-aware scheduling, carbon-intensity classification	Git-diff change-impact analysis with Maven module mapping; LightGBM carbon-intensity classifier; Prophet forecaster; Jenkins and GitHub Actions	11 paired Jenkins runs; time-series cross-validation against baseline models	C1 Release individual thesis

			integration		
Rathnayake D.M.B.H. (IT22149930)	C2 Deploy (SP2)	Carbon-aware deployment, deployment strategies, agentic (ReAct) decision-making	ReAct agent with five tools; Green Score; strategy selection and retry loop; Jenkins Shared Library; energy profiler with per-phase carbon snapshots	35 deployment records, 59 carbon reports, 30 agent decisions, 103 carbon snapshots	C2 Deploy individual thesis
Thassara M.P.M. (IT22281432)	C3 Operate (SP3)	Predictive resource management, Kubernetes autoscaling, carbon-aware workload scheduling	LSTM forecasting engine; Carbon Estimation Engine; Job Priority Engine; Runtime Decision Engine	Held-out Bitbrains fastStorage evaluation (R^2 of about 0.95, MAPE 3.28%); functional comparison with the Horizontal Pod Autoscaler	C3 Operate individual thesis
Kodithuwakku I.P. (IT22189776)	C4 Monitor (SP4)	Sustainability monitoring, energy and carbon estimation, anomaly detection	CPU-based energy and carbon estimation; stage-specific baselines; Z-score and Isolation Forest detection; Sustainability Health Score; Prometheus and Grafana dashboards	63 stage-level records from 21 Jenkins executions; 8 statistical and 6 Isolation Forest anomalies	C4 Monitor individual thesis

4.5.2. Shared and integration work

Table 4.8 lists the work that belongs to the whole team. A named lead is accountable for each item, even where all members contributed.

Table 4.8 Shared and integration tasks

Task	Lead	Contributors	Output or evidence
Overall problem definition, research gap and group objectives	Shared (All)	All	Chapters 1 and 2
High-level architecture and end-to-end data flow	Shared (All)	All	Figure 3.1; Section 3.7
Shared carbon-intensity signal and fallback (I1)	Shared (All)	C1 to C4 owners	Shared client; Appendix A
Release → Deploy interface (I2)	C1/C2 owners	C1 and C2 owners	Appendix A; integration tests
Deploy → Operate interface (I3)	C2/C3 owners	C2 and C3 owners	Appendix A; integration tests
Records to Monitor (I4) and feedback (I5)	C4 owner	C4 owner with C1 to C3 owners	Appendix A; Monitor dashboards
Integrated environment on the Oracle VM (K3s, Jenkins, language model, Prometheus and Grafana)	Shared integration	C1-C4 owners	Section 5.1 and 5.3
End-to-end integration testing	Shared (All)	All	Section 6.2
Consolidated evaluation and overall results	Shared (All)	All	Section 6.3
Common report and journal paper	Shared (All)	All	This report; the journal manuscript

4.5.3. Dependency matrix

Table 4.9 shows what each component needs from the others. Each row is a consuming component and each column is a provider. The interface codes (I1 to I5) are the same as in Table 4.1.

Table 4.9 Dependency matrix (row depends on column)

Consumer ↓ / Provider →	C1 Release	C2 Deploy	C3 Operate	C4 Monitor	External carbon service
C1 Release	n/a	Strategy pre-selection by the Green AI agent in the pre-flight	None	Baselines and anomaly feedback	Live intensity, history and forecast data (I1)

		phase		(I5)	
C2 Deploy	Scoped build artefacts, carbon context and pre-selected strategy (I2)	n/a	None	Baselines and anomaly feedback (I5)	Live intensity and live check before promotion (I1)
C3 Operate	None	Deployment profile and per-phase carbon snapshots (I3)	n/a	Baselines and anomaly feedback (I5)	Live intensity for carbon estimation (I1)
C4 Monitor	Execution records and telemetry log (I4)	Deployment records, carbon reports and snapshots (I4)	Runtime records (I4)	n/a	Carbon intensity for emission estimates (I1)

The matrix has three consequences for how the team worked.

Direction of dependency. The main chain runs from Release to Deploy to Operate, and Monitor depends on all three (Section 2.5.4, D3). Interface agreements therefore had to be fixed before dependent components could be integrated and tested (Section 4.1). The pre-flight consultation of the Green AI agent by Release adds a link in the opposite direction, so the Release and Deploy owners share ownership of I2.

Shared single points of dependence. All four components depend on the external carbon service (D1). The shared client and fallback (I1) are therefore the one task that needed all four members to agree.

Delay propagation. A delay in Release would block Deploy integration testing, and Monitor could not complete its evaluation until Release, Deploy and Operate were producing records. Monitor was therefore the most exposed to upstream delays, and its evaluation used the records collected from real pipeline executions.

4.5.4. How contributions were verified

Contributions were recorded from. Section 5.4 presents this evidence and Appendix B records the agreed declaration of individual and shared contributions.

5. INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

The four components share one deployment platform, one carbon-data source and one telemetry stack, and each adds its own libraries on top. Using a common base kept the integration points simple (Section 4.1), because the components exchange parameters, files and metrics through tools that every member already had to use. This section describes the shared platform, the technology each component adds, the repositories and data stores, and the reasons for the main choices.

5.1. Common technology stack and infrastructure

Shared platform and infrastructure

The integrated prototype runs on a single Oracle Cloud Infrastructure virtual machine with Ubuntu Linux. Table 5.1 lists the shared layers.

Table 5.1 Shared platform, services and tooling

Layer	Technology	Used by	Role in the framework	Reason for choice
Compute	Oracle Cloud VM, Ubuntu Linux	C1 to C4	Single integrated environment for build, deployment, runtime and monitoring	One environment makes integration and evaluation reproducible;
Container orchestration	K3s (lightweight Kubernetes)	C2, C3, C4	Target of deployments and scaling decisions; hosts runtime workloads	A Kubernetes distribution small enough to run on one VM while keeping the standard Kubernetes interface
CI/CD	Jenkins; GitHub Actions workflow supported	C1, C2, C4	Runs pipelines; Release is invoked as one pipeline step and Deploy as a Shared Library	Widely used CI platform; the integration needs only a pipeline step or a library call
Build system	Apache Maven (multi-module)	C1, C4	Defines modules, builds and tests that Release scopes	Provides the module and reactor structure that change-impact analysis relies on
Carbon-intensity data	Electricity Maps API (zone LK)	C1 to C4	Live grid carbon-intensity signal for the Sri Lankan grid, with history and forecasts	One external signal shared by all stages (I1); historical heuristics take over during outages
Local language	Ollama	C2 (and the	Runs the language model behind the	Keeps pipeline data inside the environment

model		Release pre-flight call)	Deploy agent on the same host	and avoids per-call external AI costs
Telemetry and dashboards	Prometheus and Grafana	C3, C4	Collect runtime metrics; display baselines, anomalies and the Sustainability Health Score	Standard open-source telemetry tools for Kubernetes
Version control	Git; GitHub	C1 to C4	Source control, change history, published artefacts	Git history is also the input to Release's change-impact analysis

Technology added by each component

Table 5.2 lists what each component adds to the shared base.

Table 5.2 Component-specific technology

Component	Language and form	Main models and libraries	Data used
C1 Release	Self-contained Python package, configured through a YAML file	LightGBM classifier (39-feature vector with lag and rolling-window features); Prophet forecaster; Optuna for tuning; SHAP for feature attribution	Hourly Sri Lankan grid carbon-intensity data; paired Jenkins execution records; git diffs
C2 Deploy	Jenkins Shared Library with an agent service	ReAct agent with five tools; locally hosted language model through Ollama; weighted Green Score; rule-based fallback	Live intensity, system resources, time context, deployment history, per-phase carbon snapshots
C3 Operate		LSTM forecaster; Carbon Estimation Engine; Job Priority Engine; Runtime Decision Engine	Bitbrains fastStorage traces; Kubernetes runtime telemetry
C4 Monitor		CPU-based power and energy model; Z-score baselines; Isolation Forest per stage; Sustainability Health Score	Stage-level Jenkins records (duration, CPU, overhead, energy, carbon, intensity)

Repositories and data stores

The carbon-intensity classification dataset, training scripts, trained models, evaluation scripts and paired Jenkins execution records are published in the project's public repository (release v1.0-submission), which lets the reported results be checked.

Data is held in the simplest form that each interface needs. Release is configured through a YAML file and writes completed runs to a CSV telemetry log, with rescheduled and aborted builds excluded. Deploy keeps a deployment history, carbon reports and per-phase carbon snapshots. Runtime metrics are held by Prometheus, and Monitor stores its historical sustainability records and baselines. Field-level formats are specified in Appendix A.

Environments

Components were developed individually and then integrated and evaluated on the Oracle VM described above. Using the same environment for integration and evaluation keeps the reported results consistent with what was built, but it also means the evidence comes from a single-VM, single-cluster setting (Section 2.5.1).

Reasons for the common choices

Four principles guided the shared technology choices.

Open-source and low cost. The stack uses open-source components throughout, apart from the external carbon-intensity service. The running cost of the prototype is therefore small, and any team with Jenkins and Kubernetes can reproduce it (Section 7.3).

Few moving parts at the interfaces. Release is a self-contained package invoked through a single pipeline step, which avoids inter-service network calls. Deploy is a single library call. This keeps adoption effort low and reduces the number of failure points between components (Section 4.3).

Local processing where possible. The language model is hosted on the same host, so pipeline data is not sent to an external AI service. The only external call is for carbon-intensity data, with a historical fallback if the service is unavailable.

Standard tools for observability. Prometheus and Grafana were chosen because they are the usual telemetry stack for Kubernetes, which lets Operate and Monitor use existing metrics instead of introducing a custom collection layer.

The main limitation of the stack is that it was tested only on this configuration. Other build systems, CI platforms, clouds and grid zones are outside the evaluated scope (Table 2.5), and version-specific compatibility with other environments was not tested.

5.2. Development and integration process

The team worked in parallel on four components that depend on one another (Section 2.5.4). The development process was therefore organised around keeping the hand-offs between components stable while each member developed their own stage.

Planning and iteration.

Interface-first work. Interface agreements were treated as a prerequisite for integration (Section 2.5.4, D3). Before components were joined, the team agreed what each hand-off carries: scoped build outputs, carbon context and a pre-selected strategy from Release to Deploy; deployment profiles and carbon snapshots from Deploy onwards; and execution, deployment and runtime records to Monitor (Section 4.1). Each producer documents its contract (Appendix A). This

allowed members to develop and test against agreed formats before the neighbouring component was finished.

Version control. Source was managed with Git and GitHub. The shared Jenkins library, the demonstration application used to exercise the pipeline, and the carbon-intensity classifier, with its datasets, trained models, scripts and execution logs, are kept in separate repositories, and the classifier repository is public (release v1.0-submission). Repository history and commit metadata provide traceability for component and integration work; branching and pull-request details should be verified directly from the repositories if they are required as submission evidence.

Reviews.

Continuous integration. The integrated pipeline is the project's own integration environment. A GitHub push triggers Jenkins through a webhook, and the pipeline definition (one Jenkinsfile with the shared `green-agent` library) runs Release, Deploy and Monitor together. Builds are serialised, so only one runs at a time. Two parameters were useful for safe integration testing: a dry-run mode, which runs the optimiser without building, testing or deploying, and a force-full-build mode, which bypasses selective execution for comparison runs. The framework was therefore integrated and tested by running it on a real pipeline, not on stubs.

Testing practice. Component behaviour was verified by running real pipeline executions and inspecting the console logs, decision logs and stored records. The failure paths, such as the fallback from the LightGBM classifier to the rule-based engine, the carbon-service fallbacks and the monitoring cancel path, were verified manually in pipeline logs and have no automated test cases (Section 6.6.4).

Issue management and release practices. The application image is tagged with the build number and with the latest and canary tags at each build, and the evaluation artefacts were frozen in a tagged release of the public repository (v1.0-submission) so that the reported results can be reproduced.

5.3. Deployment and operational configuration

This section explains how the integrated solution is built, configured, deployed, monitored, and recovered. It describes the prototype on its single Oracle Cloud virtual machine (Section 5.1).

Build. The target application is a Maven multi-module project. The pipeline builds and tests only the modules affected by a change, and builds a Docker image of the application only when the application module is affected. The optimiser itself is distributed as a Docker image that contains the bundled LightGBM model, and the pipeline streams the source tree into it for analysis. A setup stage checks for Docker, Docker Compose and Maven on the build agent and installs any that are missing.

Deployment. The pipeline deploys the image with the strategy chosen by the Green AI agent. The three strategies behave as follows.

- **Recreate** replaces the running instance directly.
- **Rolling** replaces containers one at a time, with a 15-second wait between replacements.
- **Canary** runs a new instance beside the stable one, checks its health (up to 12 attempts, 5 seconds apart), observes it for 60 seconds, and fails the deployment if more than five errors occur. The canary container is removed on failure.

Carbon snapshots are recorded at three phases (before deployment, with the canary live, and after promotion).

Configuration. Table 5.3 lists the main configuration items and where they are set.

Table 5.3 Main configuration items

Item	Set by	Purpose
Release YAML configuration	Release	Git base and head references, shared modules, rules (for example, skip non-code changes and global trigger files), Maven goals, carbon provider and zone (Sri Lanka), minimum history and backfill
Pipeline environment variables	Jenkinsfile	Deterministic defaults (for example, deployment strategy defaults to rolling), set before any stage runs
Pipeline parameters	Jenkins	Dry run, force full build, green scheduling on or off, and the pre-selected strategy
Commit-message flags	Developer	`[urgent]` skips all green checks; `[bypass-green]` skips the scheduling delay
Service endpoints and paths	Jenkinsfile	Green AI agent service on the host-to-Docker bridge address, Monitor installation directory, container image names
Carbon-service access	Release, Deploy, Monitor	Electricity Maps zone and API key
Monitor session arguments	Jenkins to Monitor	Stage, pipeline, run identifier, sampling interval, zone and status

Monitoring. Jenkins notifies on the start and end of a run. Monitor opens a sampling session for the release and deploy stages, stores one lifecycle record per stage in MongoDB, and displays the stored data on its Flask-based dashboard. Operate's CPU metrics are read from Prometheus. The Green AI agent keeps a decision log (latest 500 entries) and the Release stage appends one row per completed build to a metrics file. Rescheduled and aborted builds are excluded from that file.

Backup and restore. No formal backup procedure was implemented for the prototype. Table 5.4 shows how each kind of state can be recovered.

Table 5.4 State and recovery

State	Held in	Recovery
Source code and configuration	Git repositories	Clone from GitHub
Trained classifier and optimiser	Bundled in the optimiser Docker image; published in the project repository	Pull the image or the tagged release
Carbon history cache	Local file on the build agent	Rebuilt by backfill from the carbon service (the free tier returns 24 hours, so older hours are padded with synthetic values)
Lifecycle records	MongoDB, with a local CSV fallback if the database is unreachable	
Deployment history and decision log	SQLite database and JSON file on the host	
Release metrics	CSV file on the Monitor host	
Jenkins configuration and credentials	Jenkins home on the VM	

Rebuilding the environment from the repositories and images would restore the software, but not the historical records or Jenkins state, unless those are copied separately. This is a limitation for any use beyond the prototype (Section 6.6).

5.4. Evidence of team execution

This section summarises the evidence that the team planned, built and integrated the solution. The evidence is summarised here and not reproduced, and the supporting records are available on request and are cited in Appendix B.

Table 5.5 Sources of execution evidence

Source	What it shows	Summary
MS Planner	Task allocation, ownership and completion by component and phase	
Microsoft Teams	Communication, supervisor meetings and decisions on interfaces	
Git history	Individual and shared development activity, and integration commits	Repository commit history, merged changes, integrated pipeline execution records, and versioned submission artefacts.
Meeting records	Decisions on architecture, interfaces and scope	
Research diaries	Individual research and reasoning over time	

Jenkins and monitoring records	That the integrated pipeline ran repeatedly on real executions	21 monitored pipeline executions giving 63 stage-level records; 11 paired Release runs; 35 deployment records; 30 agent decisions

Table 5.6 Integration milestones.

Milestone	Date	Evidence
Interface agreements fixed (Appendix A)		
Release and Deploy joined in one pipeline (I2)		
Deployment records flowing to Monitor and Operate (I3, I4)		
First end-to-end run of the integrated pipeline		
Evaluation data collection		
Frozen release for submission (v1.0-submission)		Tagged release in the public repository

What the evidence shows. The contribution evidence is consistent with the component ownership recorded in Table 4.7 and with the shared integration responsibilities described in Table 4.8. Repository history, pipeline execution records, evaluation datasets, and the individual theses provide complementary evidence of implementation and testing. The integrated results reported in Chapter 6 should be interpreted together with the stated prototype limitations and the dependency risks identified in Section 6.6.

6. INTEGRATED TESTING EVALUATION AND VALIDATION

6.1. Evaluation framework and success criteria

6.2. Integration and end to end testing

6.3. Overall results

6.4. Stakeholder and user validation

6.5. Achievement of common objectives

Each group objective in Table 2.4 is judged against the evidence in Sections 6.2 to 6.4 and the success criteria in Section 6.1. An objective is fully achieved when every clause is supported by evidence, partly achieved when at least one clause is supported and at least one is unmeasured or shown only functionally, and not achieved when no supporting evidence exists. Table 6.5 gives the judgement for each objective, and the text after it summarizes the overall position.

Table 6.1 Achievement of the group objectives

Objective	Evidence	Status	Justification
GO1 (C1): reduce unnecessary build and test execution and the carbon cost of necessary execution	11 paired Jenkins runs: mean pipeline time reduction 45.3% (median 32.0%, Wilcoxon p = 0.0049, bootstrap 95% CI 23.2% to 67.9%); 91.3% of test executions skipped (13 of 149); analysis overhead about one second per run; classifier macro F1 of 0.9787 against 0.9597 for the best baseline	Partly achieved	The reduction of unnecessary execution is well evidenced, although uneven (two single-module runs were slower). The carbon saved by deferring necessary work was not measured, and the forecaster used to find a lower-carbon hour had a MAPE of about 51%
GO2 (C2): measure deployment energy and carbon and select low-carbon windows and strategies within bounded delay	35 deployment records, 59 carbon reports and 103 per-phase carbon snapshots; 17 deploy and 13 wait decisions, with 11 of the 13 waits followed by a greener window; mean decision score 78.2 out of 100	Partly achieved	Measurement and carbon-sensitive timing were demonstrated. The evaluation did not compare the agent's choices with a carbon-agnostic baseline, so a reduction in emissions is not established. Canary used about 70.8% more carbon than recreate, which is a cost of its reliability benefit. Delay is bounded to one deferral in the integrated pipeline, but the agent calls have no timeout

			(Section 6.6.5)
GO3 (C3): forecast Kubernetes demand and integrate carbon estimation and SLA-aware prioritisation into scaling	R^2 of about 0.95 and MAPE of 3.28% on held-out Bitbrains traces; the decision engine produced scale-up, scale-down, maintain and delay recommendations; functional comparison with the Horizontal Pod Autoscaler	Partly achieved	Forecasting is well evidenced. The carbon-aware and SLA-aware decisions were shown functionally on recorded traces, not on a live cluster, and no emissions or SLA outcomes were measured
GO4 (C4): establish stage-specific baselines, anomaly detection and a consolidated indicator	21 pipeline executions giving 63 stage-level records; per-stage baselines; 8 Z-score and 6 Isolation Forest anomalies; Sustainability Health Score; dashboard	Fully achieved	The capability was built and run on real pipeline records. This judgement concerns establishing the capability, not its accuracy: no labelled anomalies exist, so detection accuracy is unmeasured and the validity of the score is untested
GO5 (C1 to C4): integrate the components with a shared carbon signal, defined interfaces and a feedback path	Release, Deploy and Monitor run in one Jenkins pipeline; Release passes strategy and carbon context to Deploy; records flow to Monitor; interfaces documented in Appendix A; integration and end-to-end test evidence reported in Section 6.2	Partly achieved	The forward chain and record collection are implemented. The components share a data source, but each has its own carbon client and fallback. The feedback path from Monitor to the earlier stages is designed only and does not exist in the implementation
GO6 (all): evaluate the aggregate reduction in unnecessary computation and carbon while preserving velocity, performance and SLA compliance	Release reduced compute, time and (by implication) cost together; overhead about one second; deferral limited to one per change; Monitor measured 0.001163 kWh and 0.000502 $kgCO_2eq$ as a proof of concept	Partly achieved	Reduction of unnecessary computation is supported at Release only. No experiment measured aggregate emissions across the lifecycle. Application performance and SLA compliance were not measured under live load, and deferral adds delay by design

Overall judgement. One objective is fully achieved, five are partly achieved, and none is wholly unachieved. The strongest evidence is for the Release and Monitor components, and the weakest is for aggregate effects, because the project demonstrates that the four mechanisms work together but does not measure what they save in total. These judgements are consistent with the feasibility-level nature of the evaluation (Section 6.6).

The following would move the partly achieved objectives to fully achieved:

- an end-to-end run compared with a carbon-agnostic baseline on the same commits;
- a live-cluster evaluation of Operate, with SLA and emissions metrics;
- an implemented and measured feedback path;
- a single shared carbon client with one fallback.

6.6. Integrated limitations and threats

6.6.1. System-level limitations

The closed loop does not yet exist in the implementation. Monitor detects anomalies and computes health scores, but nothing returns that information to Release, Deploy or Operate, and Jenkins ignores Monitor's exit codes. The framework is therefore a forward-chained pipeline with an observer attached, and the benefit of feeding monitoring back into decisions is untested.

No end-to-end measurement of emissions. Each component was evaluated on its own data. No experiment ran all four stages on the same changes and compared the total energy and carbon with a conventional pipeline. Aggregate claims are therefore inferred from component results, not measured.

Operate was evaluated offline. The forecasting and decision logic were assessed on recorded Bitbrains traces, so the effect on a running cluster, including SLA compliance, is not known.

The framework's own energy cost is only partly known. The Release overhead is measured (about one second per run). The energy used by the language model, the forecasting and classification models, the monitoring sampler and the dashboard was not measured, so the net saving is not established.

Estimates and defaults carry the results. Energy is measured from hardware counters where Monitor can read them and estimated from CPU utilisation otherwise, and the Deploy profiler uses fixed server specifications. Carbon values depend on an external service and on component-specific fallbacks.

6.6.2. Dependencies between components

Table 6.2 lists the dependencies that link the components and what happens when each fails.

Table 6.2 Inter-component dependencies and their consequences

Dependency	Used by	Consequence of failure	Current protection	Remaining gap
External carbon-intensity service	All four	Decisions made on stale or default values	Fallbacks in each component	Fallback values and units differ; no retry in Release and Monitor; staleness not checked

Green AI agent service	Release pre-flight and the strategy confirmation	Strategy defaults to rolling; no delay	Default strategy; warning logged	No timeout, so a hung agent hangs the build
Jenkins pipeline as the shared backbone	Release, Deploy, Monitor	Pipeline failure stops all stages	One build at a time; failure notifications	Single point of failure; Release and Deploy are coupled in one pipeline file
Release outputs	Deploy	Deploy receives defaults or no image	Default values; pipeline stops on build failure	Defaults mask a Release failure from Deploy
Monitor command line	Release, Deploy	No lifecycle record	Failures caught; pipeline continues	Records are silently lost; the sampler may be orphaned after a hard abort
Single host (VM with K3s)	All four	Total loss of service	None	No redundancy or multi-node testing

6.6.3. Threats to validity

Internal validity. Energy figures mix measured and estimated sources, and the carbon fallbacks differ between components (for example, 300 gCO_2eq/kWh in Release and 500 in Monitor), so comparisons across stages are approximate. Hours 25 to 168 of the carbon history are synthetic when only the free API tier is available, which weakens the long-lag features of the classifier. Time-series cross-validation with a 24-hour gap was used to avoid leakage, because lag and rolling-window features supply 96.2% of the classifier's importance. About 70% of agent decisions occurred under high-carbon conditions, which limits the variety of situations tested.

External validity. The evidence comes from one VM, one K3s cluster, Maven projects on Jenkins and one grid zone (Sri Lanka). The classifier was trained on Sri Lankan grid data and would need retraining elsewhere (Assumption A4). The Bitbrains traces describe data-centre virtual machines and may not match the behaviour of other container workloads. The Deploy agent uses a small locally hosted language model, and results may differ with other models. The findings should not be extended beyond these boundaries (Table 2.5).

Construct validity. The Sustainability Health Score combines chosen inputs and weights into one number, and nothing shows that it represents sustainability as a broader concept. A "green window" is defined by the classifier's labels on hourly national grid data, not by measured emissions saved. Carbon intensity is treated as the signal for when to run, although the marginal source on the grid may differ from the average reported.

Conclusion validity. The samples are small: 11 paired Release runs, 35 deployments and 63 monitoring records. The breakdowns by change size (two to six runs per category) and by deployment strategy are indicative only. Anomaly detection has no ground truth, so the eight and six flagged anomalies cannot be classified as correct or false. The results are a feasibility demonstration and not a statistically robust estimate of production impact.

6.6.4. Unresolved integration risks

Table 6.3 Unresolved integration risks

Risk	Impact	Suggested action
No timeout on Green AI agent calls	A hung agent blocks the build indefinitely, breaking the bounded-delay guarantee	Add a timeout with fallback to the default strategy
Delay limits stated in the design (eight hours; 30-minute retries for up to six hours) not confirmed in the integrated code	Documented guarantees may not match behaviour	Confirm and enforce one explicit cap, or correct the documentation
Monitor feedback not implemented	The closed loop is unverified	Implement a feedback signal and measure its effect
Different carbon clients, fallback values and units	Inconsistent figures across stages	Use one shared client and unit, with one fallback
Orphaned sampler after a hard abort; no maximum sampler duration	Resource use continues after a failed run	Add a maximum duration and cleanup on abort
Unhandled exception in Isolation Forest training, with retraining at each analysis	A lost record; training time grows with history and may approach the one-minute command timeout	Wrap training, persist the model, or retrain on a schedule
No limit on the number of reschedules	Prevented only by a flag on the rescheduled build	Add an explicit maximum reschedule count
Release telemetry appended to a CSV without locking	Rows may be lost if builds ever ran concurrently	Use an atomic write or a database
Rolling deployments have no health-check retry	A slow-starting container causes a failure	Add a retry loop
First-commit case untested; exception paths lack automated tests	Failure behaviour is verified only manually	Add automated tests for the fallback paths
Operate's failure handling	Unknown	

The most important of these are the missing agent timeout and the unimplemented feedback loop, because they affect the project's two central claims: that delay is always bounded and that monitoring improves later decisions.

7. COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1. Value proposition and beneficiaries

The central value of the framework is that a delivery team gets one carbon-aware capability across the whole lifecycle, without assembling separate tools for selective testing, carbon-aware scheduling, deployment, autoscaling and sustainability monitoring. The framework also treats delivery speed and service levels as fixed limits. Delays are bounded and urgent work proceeds at once, so adopting it does not require a team to accept slower delivery in exchange for lower emissions (Section 2.5.3).

For the primary stakeholders in Table 1.1, the value is practical. Release removes redundant build and test work, which shortens feedback loops and reduces compute cost as well as carbon. In the prototype, mean pipeline time fell by 45.3% across 11 paired runs, and 91.3% of test executions were skipped. Deploy makes the energy cost of each deployment strategy visible and places releases in cleaner periods where the delay is acceptable. Operate adds forecast-driven scaling with SLA-aware deferral of flexible jobs. Monitor gives managers a baseline-aware view and an early warning of regressions. Table 7.1 summarises who benefits and how.

Table 7.1Beneficiaries and the value created

Beneficiary	Value created	Components	Evidence
DevOps and platform engineers	Less wasted compute; carbon-aware timing with bounded delay; setup through a single pipeline step or library call	C1, C2	Chapter 6 (time reduction, overhead)
Developers	Faster feedback on commits that touch few modules	C1	Chapter 6 (selective execution); weaker for the smallest changes
Site reliability and operations engineers	Proactive scaling that protects SLAs while deferring flexible work	C3	Chapter 6 (forecast accuracy); functional evidence only
Engineering managers and sustainability officers	Baselines, anomaly alerts and a consolidated score	C4	Chapter 6 (63 stage-level records)
Organisations	Lower compute cost alongside lower emissions; an operational basis for sustainability reporting	All	Indicative only; no cost study was done
Society	Smaller footprint of software delivery	All	Intended effect, not measured at scale (Section 1.3)

7.2. Market alternatives and differentiation

Target segments. The most plausible early users are teams that already run Jenkins or comparable CI with Maven-based multi-module projects and a Kubernetes platform, such as software companies, service providers and university or research groups, particularly in regions where grid carbon intensity varies enough to make timing matter. No market sizing was carried out in this project.

Alternatives. No single product covers the same ground, but each stage has existing alternatives or substitutes (Table 7.2).

Table 7.2 Alternatives and substitutes

Alternative	What it offers	What it does not cover	Relationship
Default CI and Kubernetes behaviour (full rebuilds; Horizontal Pod Autoscaler)	The status quo in most teams: simple and widely understood	No change-scoped execution, no carbon awareness, reactive scaling, uniform treatment of workloads	Substitute (what the framework improves on)
Commercial predictive test selection (for example, CloudBees Smart Tests)	Machine-learning selection of the most relevant tests, integrated with Jenkins and other CI systems, with vendor-reported CI runtime reductions of 35–70%	Focused on test time and compute cost; carbon intensity of the grid, deployment, runtime scaling and monitoring are outside its scope	Competitor for the Release component; partly complementary
Carbon Aware SDK (Green Software Foundation)	An open-source API and command line tool that supplies carbon-intensity data from several sources, including Electricity Maps and WattTime, to help software run when energy is greener	A data and tooling layer rather than a lifecycle decision framework; it does not decide builds, deployments or scaling	Complement and possible data source for the shared signal (I1)
Carbon accounting tools (for example, Cloud Carbon Footprint) and cloud providers' own carbon reports	Estimation and reporting of emissions	Measurement and reporting without stage-level baselines, anomaly detection or decision support	Complement for the Monitor component

The facts for the second and third rows come from the vendor's and the foundation's own descriptions. The performance figure for the commercial product is vendor-reported and was not independently verified. CloudBees describes its Smart Tests as selecting only the tests relevant to each change, with customers reporting 35–70% reductions in CI runtime on their own pipelines.

The Carbon Aware SDK is described by the Green Software Foundation as a web API and command line tool that provides emissions data from sources such as WattTime and Electricity Maps, so that software can run when green energy is available. cloudbeesgreensoftware

Differentiation. The individual techniques in the framework (selective execution, carbon-aware scheduling, forecasting, anomaly detection) are established, so they are not a defensible advantage on their own. The differentiation lies in three things that the alternatives above do not combine. First, carbon awareness is applied consistently at all four lifecycle stages with explicit bounds on delay. Second, a closed feedback loop lets monitoring findings influence earlier decisions. Third, the prototype is built on open, low-cost components and a locally hosted language model, so pipeline data need not leave the organisation's environment. The honest limit of this claim is that the integrated approach has been shown to be feasible on one prototype, and has not been compared head to head with the commercial alternatives.

7.3. Business model and feasibility

Technical and operational feasibility. The evidence of Chapter 6 shows that the four components run together on a single virtual machine with K3s, Jenkins and a local language model. The analysis overhead at Release was about one second per run, and adoption needs only a pipeline step or a library call per component (Section 4.1). The remaining feasibility questions are scale, multi-grid support and long-term maintenance (Sections 2.5.1 and 6.6).

Business model options. No pricing or willingness-to-pay evidence was collected, so Table 7.3 sets out options for discussion, not a validated model.

Table 7.3 Alternatives and substitutes

Option	Description	Resources and costs	Main risk
Open-source core with paid support	Release the framework openly; charge for integration, tuning and support	Engineering time for maintenance and support; documentation	Revenue depends on adoption and on support demand
Hosted dashboard and reporting service	Offer Monitor's baselines, Sustainability Health Score and reports as a subscription	Hosting, carbon-data subscription, security and uptime obligations	Competes with cloud providers' reporting; licence terms of dashboard components (Section 7.4)
Advisory and sustainability-reporting services	Use the framework to support organisations' sustainability reporting	Domain expertise; validation against recognised standards	Needs stronger validation of the score and estimates

Resources and costs. The prototype's running costs are modest: one virtual machine, open-source software, and grid carbon-intensity data. Commercial use would add a carbon-data subscription, ongoing model retraining for each new grid or workload (Assumption A4) and support effort.

7.4. Adoption, intellectual property and compliance

Adoption barriers. Table 7.4 lists the main barriers identified from the scope and limitations of the project.

Table 7.4 Adoption barriers and mitigation

Barrier	Reason	Mitigation
Trust in automated deferral and deployment decisions	Teams may resist letting software delay a release or choose a strategy	Bounded delays, explainable decision scores, fallback to immediate execution, a passive or advisory mode
Limited evidence	Small samples, one VM and one grid zone (Section 6.6)	Pilot studies in other environments before wider claims
Platform fit	Evaluated only on Jenkins, Maven and K3s	Modular design; extension to other build systems and CI platforms as future work
Data dependency	Needs a reliable carbon-intensity source and, for each new grid, retraining	Shared client with fallback (I1); retraining guidance
Operating a local language model	Needs compute capacity and skills; outputs can vary	Rule-based fallback; monitoring of agent decisions

Ownership and licensing. Ownership of the code and report is governed by SLIIT's intellectual property policy. The prototype uses open-source software, each under its own licence. The main points to check before any commercial use are as follows.

- The licences of the core tools (for example Jenkins, Maven, K3s, Prometheus, LightGBM, Prophet and Ollama) are permissive, but.
- Grafana's licence has obligations that matter for a hosted service.
- The language model used with Ollama has its own licence terms.
- The Electricity Maps plan and terms must permit the intended use, and the terms of use of the Bitbrains traces should be checked (they were used for evaluation only).

Data protection, ethics and regulation. The framework processes operational data (changed file paths, resource utilisation, durations and grid readings), not personal data by design. If commit metadata such as author names is stored in the telemetry log, the Personal Data Protection Act, No. 9 of 2022 of Sri Lanka may apply to its handling. Hosting the language model locally avoids sending pipeline data to an external AI service. If surveys or interviews were used for stakeholder validation, state the ethics approval and consent arrangements here.

Professional responsibilities. The framework makes automated decisions about when and how software reaches production. Responsible use requires that these decisions can be explained, overridden and audited, and that failure always degrades to normal, non-carbon-aware behaviour (Section 4.4). Energy and carbon figures are estimates from CPU utilisation (Assumption A3) and should be reported as such, and the Sustainability Health Score has not been independently validated (Section 6.6). Presenting it as proof of sustainability would risk overstating the benefits.

7.5. Sustainable Development Goal contribution

Table 7.5 links the framework to specific SDG targets [19]. These are plausible contributions at pipeline and cluster level, not measured national or global effects (Section 1.3).

Table 7.5 Contribution to selected SDG targets

SDG and target	Plausible contribution	Possible negative effect	Needed to sustain the benefit
13.2 Integrate climate change measures into policies and planning	Makes carbon intensity an operational decision input in software delivery	Estimates may overstate savings	Report net savings, including the framework's own overhead
12.6 Encourage companies to adopt sustainable practices and sustainability reporting	Baselines and a score give organisations data for sustainability reporting	The score may be treated as a certification	Validate the score against recognised standards before external use
9.4 Upgrade infrastructure for resource-use efficiency	Reduces redundant compute and over-provisioning in delivery infrastructure	Gains are limited to the supported platforms	Extend to other build systems and platforms
7.3 Improve energy efficiency	Lower compute per delivered change; proactive scaling	Rebound effect: cheaper pipelines may be run more often	Track total usage as well as per-run efficiency
7.2 Increase the share of renewable energy	Shifts flexible work toward cleaner grid periods	Benefit depends on the grid and on how marginal intensity is measured	Evaluate in other grid zones

The framework itself consumes energy. The Release analysis overhead is small, but the energy used by the language model, the forecasting models and the monitoring stack was not measured, so net benefit is not yet established. Other risks include delays that affect developer productivity, which is why delays are bounded, and the risk of shifting work that could simply move load to another time without lowering emissions on grids where the marginal source does not change.

To keep the benefits real, the team recommends the following next steps: measure net energy including the framework's own components, validate in additional grid zones and organisations, publish the code and evidence openly (as the project's repository already does), and re-evaluate the Sustainability Health Score against independent measures before it is used for external reporting.

8. CONCLUSION AND FUTURE DIRECTIONS

8.1. Overall conclusion

This project set out to answer one problem: software delivery and the platforms it deploys to are governed by speed, reliability and resource availability, while the energy used and the carbon intensity of the electricity supplying it are not treated as decision inputs at any lifecycle stage (Section 2.1). The integrated Green DevOps framework shows that this can be changed within the limits that delivery teams care about. In the evaluated prototype, carbon-aware decision-making was applied at the Release, Deploy, Operate and Monitor stages. Delays were bounded, urgent work was never held back, and every dependency had a fallback to ordinary behaviour.

The evidence supports this conclusion at the level of feasibility. At Release, change-scoped execution cut mean pipeline time by 45.3% and skipped 91.3% of test executions at an analysis cost of about one second per run. At Deploy, the agent separated deploy from wait decisions according to grid conditions, and most waits were followed by a greener window. At Operate, workload demand could be forecast accurately enough (R^2 of about 0.95, MAPE 3.28%) to support proactive, SLA-aware scaling. At Monitor, stage-specific baselines and two complementary detectors flagged abnormal behaviour in real pipeline records. These results show that the four mechanisms can run together in one pipeline without compromising delivery velocity, and that is the answer to the central question of the project.

The conclusion has limits that should be stated as plainly as the results. The evidence comes from one virtual machine, one K3s cluster and one grid zone, with small samples (11 paired Release runs, 35 deployments, 63 monitoring records). Energy and carbon values are estimates from CPU utilisation. The Operate component was evaluated on recorded workload traces, not on a live cluster, and no experiment measured the aggregate emissions reduction across the full lifecycle or the effect of the feedback loop on later decisions. Group objectives GO1 to GO5 are therefore supported by component-level and integration evidence, while GO6, the aggregate reduction, is supported only in part.

8.2. Principal integrated contribution

The contribution of the project is not any one technique, since selective testing, carbon-aware scheduling, forecasting and anomaly detection are all established. It is the demonstration that these can be combined into one coherent, bounded and verifiable decision architecture, and the lessons learned from building it.

One carbon context across stages. A shared grid signal, with a common fallback, and decisions carried between stages (for example, a deployment strategy chosen at Release under the same carbon conditions and inherited by Deploy) allowed stages to act consistently instead of independently.

Bounded delay as a design principle. Treating delivery speed and service levels as constraints, with explicit limits and an immediate-run fallback, is what makes carbon-aware decisions acceptable to the stakeholders who would have to adopt them.

Fallbacks that keep delivery moving. Weak components are tolerated by design. The carbon-intensity forecaster had a MAPE of about 51%, and the agent's language model is not always available, yet the pipeline still proceeds because both degrade to non-carbon-aware behaviour.

Stage-specific behaviour. Build, Test and Deploy showed different resource and carbon profiles, which supports baselines kept per stage, and the deployment results showed that strategy choice has its own carbon cost (canary averaged about 70.8% more emissions than recreate in the evaluation).

Honest accounting of uneven benefits. Gains were large for documentation-only and no-change commits and small or negative for the smallest single-module builds, where the optimiser's overhead is a larger share of a short run. Environmental and cost gains also coincided in the Release results, since less compute means shorter runs as well as lower emissions.

Reproducible artefacts. The trained models, evaluation scripts and execution logs are published in the project repository, so the results can be checked and extended.

8.3. Future directions

The directions below are ordered by how directly they address the limitations in Section 6.6. Table 8.1 gives the priorities, and the text that follows explains the first four.

Table 8.1 Prioritised future work

Priority	Direction	Limitation addressed	Horizon
1	Larger, longer evaluation across more repositories, pipelines and grid zones, with measured net energy including the framework's own components (language model, forecasting, monitoring)	Small samples; single VM and grid zone; unmeasured overhead	Near term
2	Live-cluster evaluation of Operate against the standard autoscaler, with SLA and emissions metrics	Offline trace evaluation only	Near term
3	Validate energy estimates against direct power measurement where available	CPU-based estimation error	Near term
4	Automate and quantify the Monitor feedback loop	Feedback effect not measured	Near to medium term
5	Improve carbon-intensity forecasting	Prophet MAPE of about 51%	Medium term
6	Validate the Sustainability Health Score and study usability with practitioners	Construct validity; no user evidence	Medium term
7	Evaluate the reliability and consistency of the language-model agent, including smaller	Non-deterministic decisions; energy cost	Medium term

	models and human-in-the-loop modes		
8	Generalise to other build systems, CI platforms and multi-cluster or multi-region deployments, including spatial shifting	Scope limited to Maven, Jenkins and one K3s cluster	Medium to long term
9	Reinforcement-learning-based optimisation; embodied carbon and infrastructure lifecycle impacts	Operational emissions only; rule-based coordination	Long term
10	Pilot with an external organisation, with security hardening and a licensing review	Adoption barriers (Section 7.4)	Long term

Stronger evidence first. The most valuable next step is more and better data. A longer evaluation on several repositories and more than one grid would show how far the results generalise, and measuring the framework's own energy would establish the net benefit that Chapter 7 identifies as open. Comparing estimated energy with direct power measurement would show how far the CPU-based model can be trusted.

Live and closed-loop operation. Running Operate against a real workload, next to the standard autoscaler, would test the SLA and emissions claims that the trace-based evaluation can only support functionally. Making Monitor's baselines and anomaly reports drive Release, Deploy and Operate automatically, and then measuring the effect on later decisions, would test the closed loop that is the central idea of the framework.

Better carbon forecasting. The forecaster was the weakest result. Improved models, or forecast data from the grid data provider, would make it possible to find greener windows further ahead and could widen the situations in which delay is worthwhile.

Credibility of the score and the agent. The Sustainability Health Score needs validation against independent measures before it is used for external reporting, and the agent's decisions need systematic testing for consistency and cost, including how smaller or non-generative models compare.

In the longer term, reinforcement learning could replace some of the rule-based coordination, embodied carbon could widen the boundary of the analysis, and a pilot with an external organisation, supported by hardening and a licensing review, would test the framework against real adoption barriers.

REFERENCES

[1] C. Freitag, M. Berners-Lee, K. Widdicks, B. Knowles, G. S. Blair, and A. Friday, "The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations," Patterns, vol. 2, no. 9, Art. no. 100340, 2021, doi: 10.1016/j.patter.2021.100340.

[2] J. Malmodin, Å. Moberg, D. Lundén, G. Finnveden, and N. Lövehagen, "Greenhouse gas emissions and operational electricity use in the ICT and entertainment & media sectors," Journal of Industrial Ecology, vol. 14, no. 5, pp. 770–790, 2010.

[3] L. Belkhir and A. Elmeligi, "Assessing ICT global emissions footprint: Trends to 2040 & recommendations," Journal of Cleaner Production, vol. 177, pp. 448–463, 2018.

[4] R. Alamer and O. Alharbi, "Sustainable Devops: A systematic literature review on reducing energy footprint in continuous integration and deployment (CI/CD) pipelines," International Journal of Computations, Information and Manufacturing, vol. 5, no. 2, pp. 24–32, 2025.

[5] A. Aneggi, X. Li, and A. Janes, "PPTAMη: Energy aware CI/CD pipeline for container based applications," arXiv:2602.12081, 2026.

[6] C. Becker et al., "The Karlskrona Manifesto for Sustainability Design," arXiv:1410.6968, 2014.

[7] S. Naumann, M. Dick, E. Kern, and T. Johann, "The GREENSOFT Model: A reference model for green and sustainable software and its engineering," Sustainable Computing: Informatics and Systems, vol. 1, no. 4, pp. 294–304, 2011.

[8] E. Peltonen et al., "Rethinking computing systems in the era of climate crisis: A call for a sustainable computing continuum," IEEE Internet Computing, 2025, doi: 10.1109/MIC.2025.3566642.

[9] Kubernetes, "Horizontal Pod Autoscaling," Kubernetes Documentation. Accessed: Oct. 11, 2026.

[10] International Energy Agency, "Data centres and data transmission networks," IEA, Paris, France, 2024.

[11] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," Software Testing, Verification and Reliability, vol. 22, no. 2, pp. 67–120, 2012.

[12] "Build system aware multi-language regression test selection in continuous integration," in Proc. 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 87–96, 2022, doi: 10.1145/3510457.3513078.

[13] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud," in Proc. 22nd International Middleware Conference, pp. 260–272, 2021.

[14] A. Radovanović et al., "Carbon-aware computing for datacenters," IEEE Transactions on Power Systems, vol. 38, no. 2, pp. 1270–1280, 2023.

[15] A. Beloglazov and R. Buyya, "Energy efficient resource management in virtualized cloud data centers," in Proc. 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing (CCGrid), pp. 826–831, 2010, doi: 10.1109/CCGRID.2010.46.

[16] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, Art. no. 15, 2009.

[17] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 1, Art. no. 3, 2012, doi: 10.1145/2133360.2133363.

[18] Green Software Foundation, “Software Carbon Intensity (SCI) Specification,” ISO/IEC 21031:2024, 2024.

[19] United Nations, “Transforming our world: The 2030 Agenda for Sustainable Development,” United Nations, New York, NY, USA, 2015.

[20] Google, “Using demand response to reduce data center power consumption,” Google Cloud Blog, 2023.

[21] M. Mota-Cruz, J. H. Santos, J. F. Macedo, K. Velasquez, and D. P. Abreu, “Optimizing microservices placement in the cloud-to-edge continuum: A comparative analysis of app and service based approaches,” arXiv:2412.01412, 2024.

[22] Kubernetes, “Deploy a release using a canary deployment,” Kubernetes Documentation. Accessed: Oct. 11, 2026.

[23] R. Alamer and O. Alharbi, “Sustainable Devops: A systematic literature review on reducing energy footprint in continuous integration and deployment (CI/CD) pipelines,” International Journal of Computations, Information and Manufacturing, vol. 5, no. 2, pp. 24–32, 2025.

[24] D. A. Herati, M. C. Aderne, and F. Kon, “The road to Sustainable DevOps,” arXiv:2503.08845, 2025.

[25] S. Shen, V. van Beek, and A. Iosup, “Statistical characterization of business-critical workloads hosted in cloud datacenters,” in Proc. 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), pp. 465–474, 2015, doi: 10.1109/CCGrid.2015.60.

APPENDICES

Appendix A Integrated interface specification

The integrated framework uses five principal hand-offs. I1 is the shared carbon-intensity context used by all four components. I2 passes Release outputs to Deploy, including scoped build artefacts, carbon context, scheduling outcome, and the pre-selected deployment strategy. I3 passes the deployment profile and carbon snapshots from Deploy to Operate. I4 carries Release, Deploy, and Operate execution records to Monitor. I5 returns stage-specific baselines, anomaly reports, and the Sustainability Health Score from Monitor to the earlier stages. These interfaces are intentionally small and loosely coupled so that each component can run independently and fall back to ordinary delivery behaviour when an external dependency is unavailable.

Appendix B Team contribution declaration

Primary ownership is divided by lifecycle stage: Mendis J.D.L. (IT22110084) - Release; Rathnayake D.M.B.H. (IT22149930) - Deploy; Thassara M.P.M. (IT22281432) - Operate; and Kodithuwakku I.P. (IT22189776) - Monitor. Shared work includes the overall problem definition, high-level architecture, interface agreements, integrated environment, end-to-end testing, consolidated evaluation, and preparation of the common report. Detailed component responsibilities and evidence are recorded in Section 4.5. Final signatures and dates, where required by the submission process, should be completed by the team members themselves.